

Overview Introduction

Portfolio 1

The first portfolio presents a comprehensive analysis of vegetation trends using GIMMS monthly NDVI data from 1982-2022 across Europe and Corine landcover classes. It examined vegetation to analyze and assess the greening trend hypothesis, seasonal variability and Corine landcover classes which accounted for the majority of vegetation changes across Europe (1982-2022).

Portfolio 2

In the second part of the portfolio, we focus mainly on analysing the changes in Land cover (LULC) with the use of achieved satellite data (Gimms_1982-2022) with Corine data (CLC 2018) and statistical methods. We used the Syndrome mapping approach as described by Stellmes et al, (2013). to detect, classify, and spatially represent patterns of land change processes by combining remote sensing time series with socio-economic patterns and climatic information. We also extract Phenology metrics after fitting the Fourier model to generate our analysis.

Portfolio 3

In the third part of the portfolio, we analyze the rainfall–vegetation relationship in Botswana using MODIS NDVI (2000–2016) and CHIRPS rainfall datasets. The study employs Distributed Lag Models (DLMs) to examine the delayed impact of rainfall on vegetation growth, revealing significant relationships at lags 0 and 1 months ($R^2 = 0.198$, $p < 0.05$), indicating a short-term vegetation response to precipitation. Furthermore, a Random Forest regression model identified key environmental variables—aridity, elevation, land surface temperature (LST), and evapotranspiration—as the dominant factors influencing vegetation variability, collectively explaining over 75% of the model variance. Overall, this portfolio demonstrates the strength of combining time-lag modelling and machine learning approaches to better understand rainfall–biomass interactions and ecological responses in semi-arid ecosystems.

Portfolio Part I: Trend analysis of NDVI data (GIMMS-archive) for Europe (1982-2022)

1.1. Introduction

This section of the portfolio focuses on the trend analysis of GIMMS (the Global Inventory Modelling and Mapping Studies) monthly-NDVI (Normalised Difference Vegetation Index) data for Europe from 1982 to 2022. The monthly NDVI values provided by this dataset are crucial for examining changes in vegetation patterns throughout time. Several environmental factors including natural and anthropogenic activities have a significant effect on vegetation and analyzing trends in NDVI (Normalized Difference Vegetation Index) is one of the effective ways to detect vegetation changes. The NDVI is calculated using measurements of how sunlight is reflected in the red and near infrared part of the spectrum and can detect vegetation (NDVI increase or decrease) when areas become greener or browner ¹.

Our study focused on understanding the impact of anthropogenic and natural processes which affected our environment. A clear picture for this can be seen through vegetation change which acts as an indicating factor for these kinds of processes. The analysis of these trends using vegetation indices were important to understand the concept. Normalised Difference Vegetation Index (NDVI), well known for its effectiveness in vegetation monitoring over larger dataset and time periods was primarily used in this study. We applied the Seasonal Mann Kendall (non-parametric test for evaluating non-linear trends) test to get the direction and strength of vegetation trend across the NDVI time series ².

This study aims to investigate changes in vegetation by analyzing NDVI trends to address our four research questions. First, it investigates whether there is evidence for the “greening trend” hypothesis in natural vegetation in relation to climate change using the full temporal period? The second question explores changes in trend strengths, if only the summer months for each year are included in the analysis (April- September)? Then it evaluates whether there is evidence for a slowdown of greening trends in the last years? Finally, the study identifies which land-use/cover class (CORINE) contributes most to the effect of a greening trend? Our results showed different NDVI trends for various regions across Europe reflecting a strong interaction between climate change, vegetation dynamics and effect of land use change practices towards stronger greening trends in certain parts of Europe.

1.2. Materials, methods, study area

The GIMMS-NDVI dataset provides one of the longest continuous global records of vegetation productivity, making it a valuable resource for monitoring long-term ecological changes. It originates from the Global Inventory Modelling and Mapping Studies (GIMMS) and includes NDVI data in raster format, derived from satellite observations³. NDVI (Normalized Difference Vegetation Index) is a widely used remote sensing indicator for assessing vegetation health and density. It is calculated using reflectance values from the near-infrared (NIR) and red spectral bands of sunlight, based on the following formula:

$$NDVI = \frac{NIR - Red}{NIR + Red}$$

In this equation, NIR represents reflectance in the near-infrared range, which is strongly reflected by healthy vegetation, while Red represents reflectance in the red spectrum, which is mostly absorbed by chlorophyll⁴. The resulting NDVI values range from -1 to +1, where higher values indicate dense, healthy vegetation, and lower values reflect sparse or degraded plant cover. The CORINE Land Cover dataset, offering comprehensive classifications of land cover across Europe, was employed to connect observed vegetation trends with particular land cover types. All analyses and data processing were carried out using RStudio. The analysis started by loading the GIMMS-monthly (1982-2022) data and then we plotted NDVI first full year layer of 1982 data so that we can visualize differences or changes through the whole year.

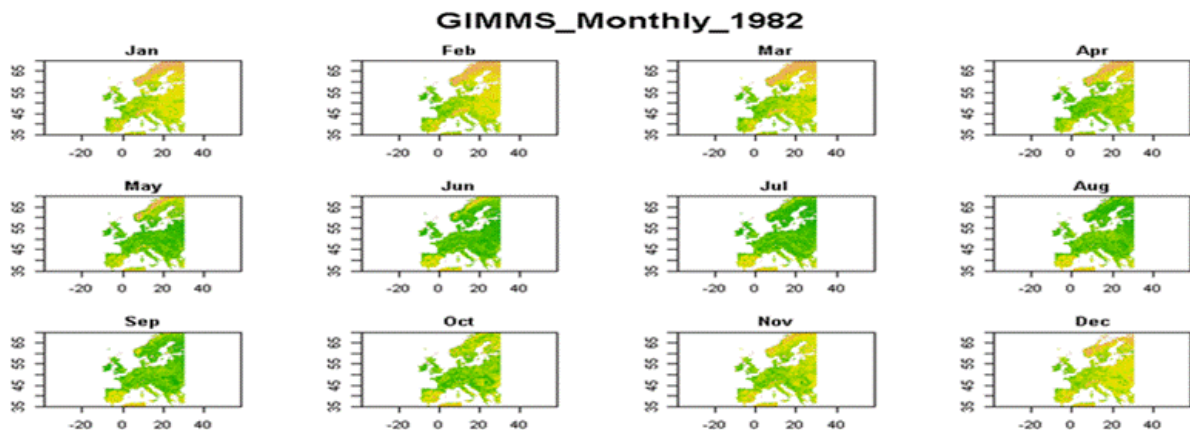


Figure 1: GIMMS_Monthly data

Later on, we converted the raster brick into a 3D array for easier manipulation. The dimension of the cube was 420,480 and 492 and we rescaled our data. After scaling most values were within normal range but some negative outliers were still there. Then we removed invalid NDVI values, replaced them with NA and also removed any no-data values (assuming -9999). Our next step was to Initialize empty arrays to store trend analysis results and then we ran the loop calculating Mann-Kendall test which took about 5 min. We converted our results arrays to raster and set spatial attributes to ensure their alignment with the original dataset and we exported it as a geo-tiff file.

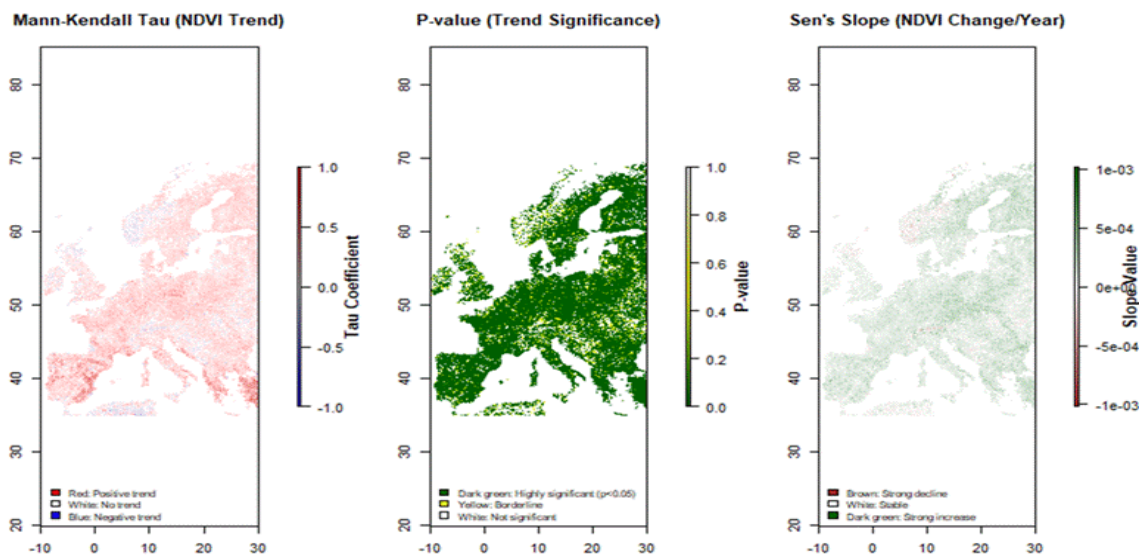


Figure 2: Mapping of the p-values for summer months

To answer the question regarding the evidence for the “greening trend” hypothesis in natural vegetation in relation to climate change using the full temporal period, long term satellite NDVI data from the GIMMS dataset (1982-2022) was analyzed. The rescaled data was aggregated into annual means per pixel to focus on long term trends. Sens slope estimator was used to quantify the direction and magnitude of vegetation change over time and applied to each pixel annual NDVI time series. We tested the statistical significance by conducting the Mann-Kendall Tau test with $p < 0.05$ indicating significant trends. Through multiplication of Sens slope by number of years, we got absolute NDVI change over 41 years. NDVI baseline from 1982 was then calculated relative to percent change with extreme values limited to $\pm 100\%$ to avoid outliers.

To investigate the changes in trend strengths if only summer months (April-September) for each year are included in the analysis, the original NDVI raster stack containing monthly layers was filtered to extract only 6 months of summer for each year (41 years * 6 months = 246 layers). Then the filtered data for summer months was converted into a 3D array for pixel wise analysis. In order to account for repetition of seasonal patterns, a Modified Seasonal Mann-Kendall test was performed, with each year treated as a block and missing values in valid time-series were linearly interpolated. The Mann-Kendall Tau, p-value and Sens slope (NDVI rate of change) were computed for each pixel and then the results were converted back into raster format and exported as geo-tiffs for further visualization and analysis. To compare the results of a full year with summer only, the slope and p-value of both rasters were aligned spatially and a slope difference raster was derived and visualized. Following that, the statistics revealed the count of pixels showing significant greening or browning trends in summer along with gain or losses in statistical significance.

Our next question was to look for the evidence for a slowdown of greening trends in the last years? To enable the comparison of long-term vegetation trends, we divided our dataset into two study periods. The first study period started from 1983-1998 and the second one was from 2007-2022. Seasonal Mann-Kendall test was applied for each pixel, resulting in tau statistics and p-values. Then they were converted back into raster format to visualize positive and negative trends for both time periods. Differences in tau between both periods were calculated to assess changes in strength of greening trends and were classified into categories of increase, slowdown and decrease. The area covered by each category was quantified to support our evidence for slowdown of greening trends in recent years.

To determine which landcover class (CORINE) contributes most to the effect of a greening trend, we aligned and reprojected the Corine Landcover classes to match the spatial extent and resolution of the NDVI dataset. We then extracted pixel values from both rasters and filtered to keep only valid pixels with greening trends. We summarized the number of greening pixels per Corine class and selected the top 20 classes and visualized their contribution. This approach helped us in evaluating which Corine landcover class contributed most to the effect of greening trends across Europe.

1.3. Results

The map shown below visualizes the tau statistics for NDVI trends (1982-2022). Dark green color showed strong greening (NDVI increasing over time), white showing no significant trend whereas bright yellow showed strong browning (decreasing NDVI over time). Overall, most of Europe indicated widespread but moderate greening trends over the years. Some Southern regions e.g. Spain and Italy showed a decreasing trend in vegetation. Scandinavia and Northern Europe showed weaker trends.

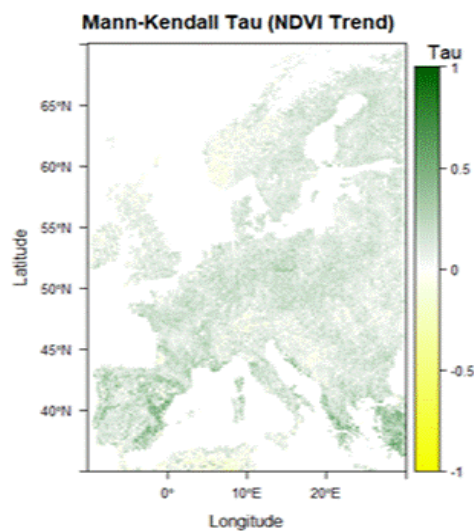


Figure 3: Mapping of the tau values

P-values from Mann-Kendall test were used to assess the statistical significance of NDVI trends (1982-2022) across Europe. P-values < 0.05 (blue color) in Central, Western and Southern Europe showed the trend was statistically significant. P-values which were closer to 1 in green regions (Northern Europe and some parts of Spain) showed non-significant trends. The values between green and blue regions showed moderate results which means the trend might exist.

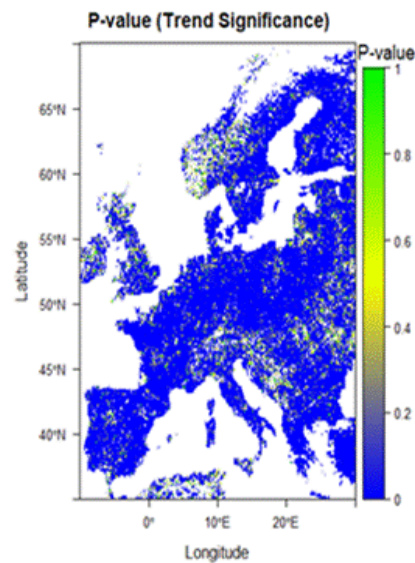


Figure 4: Mapping of the p-values

The rate of NDVI change indicates that the density of vegetation increased between 1982-2022. Some regions showed decline likely due to droughts, land-use and land degradation. The magnitude of change was small when we only looked per year but it becomes ecologically significant after four decades. The percent change in NDVI provides a clear picture of how much vegetation has changed over 41 years. 0-20 % showed slight increase, 20-40% moderate whereas above 40 % showed significant increase in vegetation. Some parts of Europe showed a loss between 0-20% supporting our analysis of vegetation loss especially in the Northern part of Europe. Our statistical summary calculated the percentage and showed 86.76% of area was greening whereas 13.23% area indicated browning.

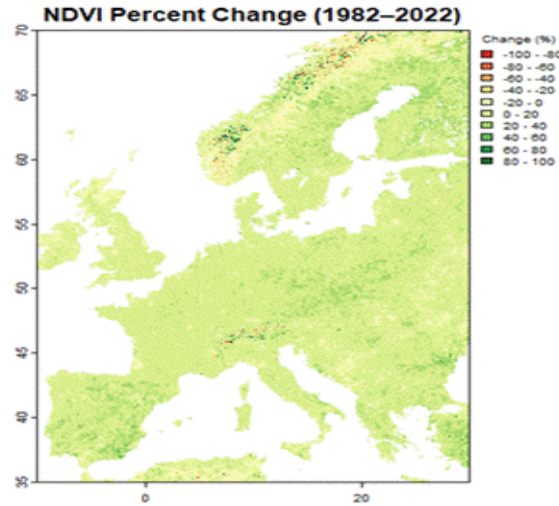


Figure 5: Percent change in NDVI

Trend analysis for summer months (Apr-Sep) of each year gave us the resulting tau and p-values. Tau values ranging roughly from -0.63 to 0.48, in which negative values indicated decreasing trend, positive values showed increased trend and values near to zero showed no trend. P-values closer to zero showed strong evidence and one means no evidence for a trend whereas slope values represented yearly change in NDVI. The maps showed the greening trend varies significantly in summer across Europe when compared to full year trend analysis.

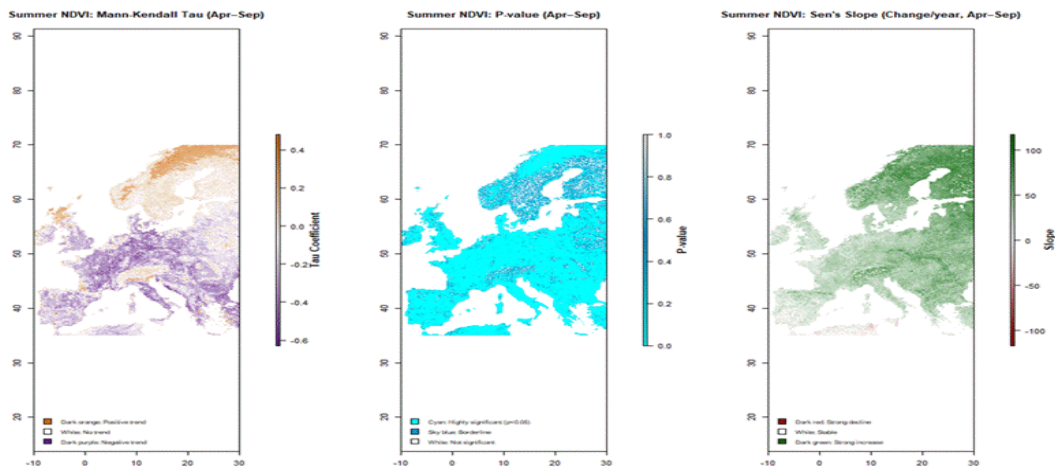


Figure 6: Mapping of the tau values for summer months

The comparison between summer and full year NDVI trends indicated that the greening trend is more pronounced in summer. 98,408 pixels exhibited a stronger positive slope in summer compared to the full year trend while 3,440 pixels showed a weaker summer slope which suggested that seasonal response is more pronounced during the growing season.

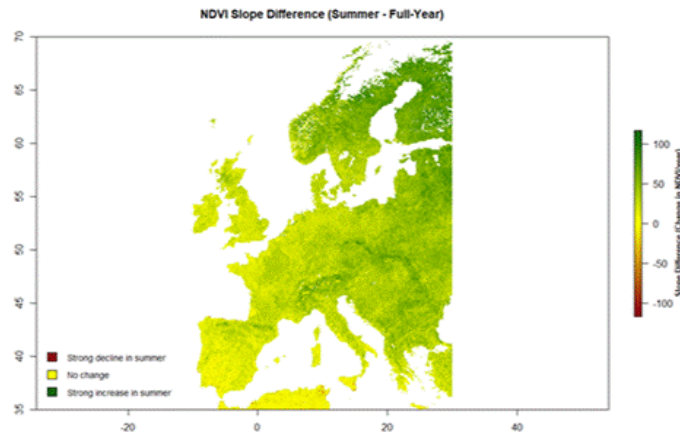


Figure 7: NDVI slope difference for the summer months

11,050 pixels gained significance during summer whereas 30,219 pixels lost significance emphasizing spatial variability in trend robustness across seasons. Overall, the greening trends are more pronounced in summer months than the annual average in 96.72% of valid pixels. Therefore, it clearly showed us that there are changes in trend strength when summer only months are included in the analysis.

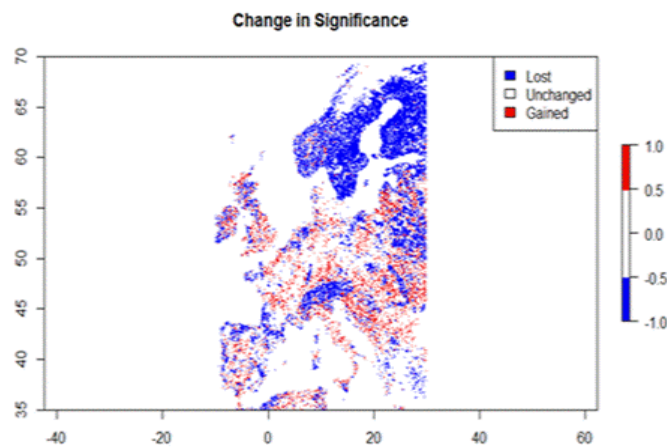


Figure 8: Change in significance

We compared the tau and p-value, after dividing our dataset into two time periods (first time period: 1983-1998, second time period: 2007-2022). The maps clearly revealed that there is evidence for a slowdown of greening trends in recent years compared to the earlier 15-year period which showed a positive greening trend. There was a negative trend in the last 15 years in countries such as Germany, Spain and France but the earlier observation of the 15-year time period in these countries had a positive trend.

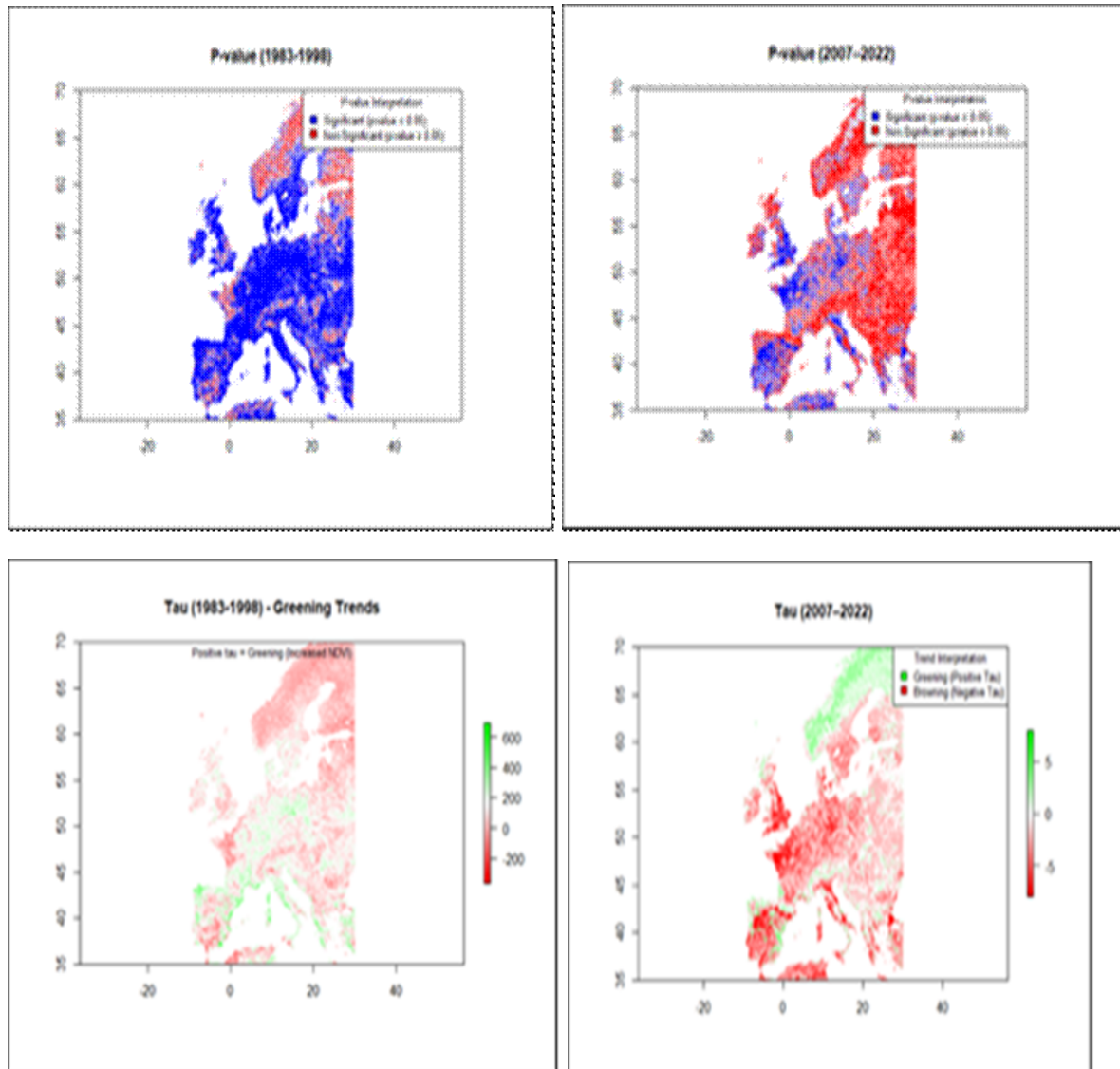


Figure 9: Mapping of the tau values for 1983-1998 (left) and 2007-2022 (right)

The bar chart showed a greening trend by landcover based on Corine level 1 landcover classes highlighting that the vegetation is more pronounced in Agricultural areas. 36,465 pixels displayed the highest number of pixels whereas forest and semi-natural areas with 35,822 pixels likely due to change in better practices which helped in vegetation health overall. Artificial surfaces showed a smaller greening trend (3,968 pixels), waterbodies (2,954 pixels) and wetlands (1,433 pixels) with the lowest count of greening pixels.

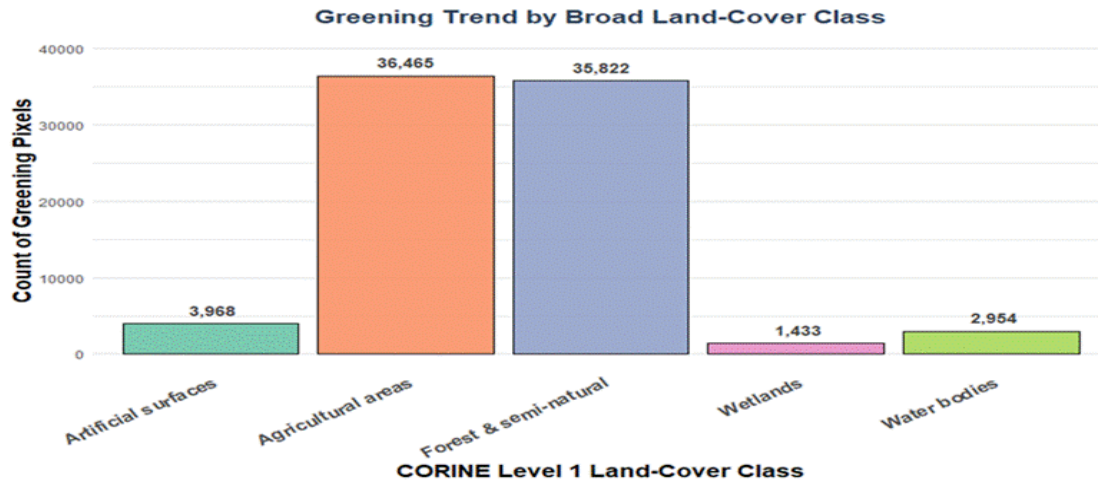


Figure 10: Greening trend per land cover class

1.4 Discussion and Conclusion

The analysis of NDVI vegetation trends across Europe using the Seasonal Mann-Kendall test showed a clear and statistically robust representation of long-term vegetation changes. The tau map indicated a persistent and widespread increasing trend in vegetation. However, certain areas displayed a browning trend, which reflected the decline in vegetation. The p-values suggested that the change was not consistently significant. Moreover, the comparison of two time periods (first period: 1983-1998, second period: 2007-2022) showed shifts in vegetation trends. Some countries showed a negative NDVI trend in recent periods, whereas positive trends of these countries in earlier periods provided clear evidence that the possible reasons could be land use change practices or climate change leading to the shifts in vegetation.

Our analysis revealed that the agricultural areas were the primary land cover class contributing most to the greening trend, followed by forest and semi-natural areas. The adoption of best practices was clearly evident through the overall vegetation health. The statistical significance of these trends represented that a greening trend occurred across certain parts of Europe from 1982-2022. The seasonal analysis of summer months reflected stronger greening trends during this time period. In conclusion, 96.72% of valid pixels exhibited a stronger positive trend compared to annual average highlighting the importance of seasonal analysis. Agricultural areas, forest and semi-natural areas, artificial surfaces, waterbodies and wetlands emerged as the top five Corine landcover classes, which played an important role in

the contribution to the greening trend. Overall, the results highlighted the prominent impact of different factors on long term changes in vegetation across Europe.

2. Portfolio Part II: Syndrome Analysis

2.1 Introduction

In this part of the portfolio, we focus on analyzing the Land use / Land cover (LULC) changes with the use of Historical satellite data (GIMMS_NDVI - 1982-2022) and statistical methods. Following the methodology described by (Stellmes et al., 2013), We employed the Syndrome mapping approach which aims to detect, classify, and spatially represent patterns of land change processes by combining remote sensing time series with socio-economic patterns and climatic information.

2.2 Materials, Methods, Study Area

The study area of this project comprised the entire European Continent, specifically, we focused on the regions covered by CORINE Land Cover (CLC) Dataset. We derived the phenological properties from monthly GIMMS_NDVI data spanning 1982-2022. We used the CORINE (CLC 2018) dataset which we recoded to classify LULC changes into categories like ‘Natural’, ‘Arable/Agriculture increasing’, ‘Agriculture decreasing’, and ‘Agriculture stable’ among others. We implemented the entire methodology using custom R scripts, and the results were then visualized in QGIS.

Fourier fit and phenological metric extraction

To capture vegetation dynamics, we fitted a Fourier polynomial model to the monthly GIMMS NDVI time series for each pixel. We applied this model annually to derive annual phenological metrics for our analysis. Some of the phenology metrics include NDVI peak months, NDVI magnitudes, and NDVI integrals. The following Fourier model was fitted in the R code program.

$$x_t = a_0 + \sum_{k=1}^{n/2} \phi_k \cos 2\pi \frac{k}{N} + \sum_{j=1}^{n/2-1} \phi_j \sin 2\pi \frac{k}{N}$$

Where :

- NDVI(Xt) is the NDVI value at time t (month)
- a0 is the mean NDVI value
- ϕ_k and ϕ_j are the amplitudes of the k-th harmonic.

- T is the period of time series (12 months of annual cycles)
- N is the number of observations in the time series

In this Fourier model, we fitted three harmonics ($N = 3$), which were sufficient to represent the vegetation cycles that we intended to capture. This we did to avoid the risk of over-fitting noise. These harmonics capture distinct aspects of vegetation dynamics:

- 1st harmonics, which represent the main seasonal cycle i.e. spring green-up and Autumn,
- 2nd harmonics capture the bi-modal patterns like double cropping or areas with 2 distinct growing seasons.
- 3rd harmonic captures finer vegetation dynamics or disturbances.

With this in mind, this model assumes that vegetation cycles repeat annually, hence ensuring its suitability for phenological studies. From the fitted Fourier model, we extracted key phenological metrics for each pixel per year. This included:

- NDVI peak month

This is basically the timing of maximum growth; the model identifies the month when the vegetation growth reaches its maximum. Here, it's also important to detect shifts in growing seasons due to climate change (cite)

- NDVI AMPLITUDE (MAGNITUDE)

This is the difference between the maximum and minimum NDVI values within a growing season, thus representing vegetation Vigor.

- NDVI Integral

This is the area under the NDVI curve, which indicates the total photosynthetic activity

Syndrome map generation and classification

In order to identify significant changes in the vegetation dynamics, we applied Mann-Kendall test to the annual time series of the derived phenological metrics (NDVI amplitude, peak month, and integrals) for each pixel across the time series in the study area. We used a significance level, which we set at 5% ($P < 0.05$), to determine statistically significant trends. We then classified the syndromes by combining the LULC information (from the recorded Corine dataset) with statistically significant trends in the phenological metrics. Specifically,

we first categorized pixels into Agriculture or Natural land use based on our recoded CORINE data. With these categories, Mann-Kendall test results for the overall NDVI trends were used to assign ‘Increasing’, ‘decreasing’, or ‘stable’ classification classes as shown in the table below:

Syndrome code	Syndrome Type
10	Agriculture - Decreasing
11	Agriculture - Stable
12	Agriculture - Increasing
20	Natural - Decreasing
21	Natural - Stable
22	Natural - Increasing

CORINE Land-cover recoding and aggregation

We recoded the Corine land cover classification (CLC 2018) dataset provided by the European Environmental Agency. The dataset has 44 LULC classes, which we first reprojected to match the projection of the GIMMS NDVI data.

We then recoded the CORINE to align with syndrome classification, specifically distinguishing between ‘Agriculture’ and ‘Natural’ land uses. Subsequently, we aggregated Corine Data to match GIMMS NDVI data (Stellmes et al., 2013) using a filter approach. This methodology recognizes that because of spatial degradation, minor or small patches of land use classes may be lost, and hence we mainly focus on major land use classes.

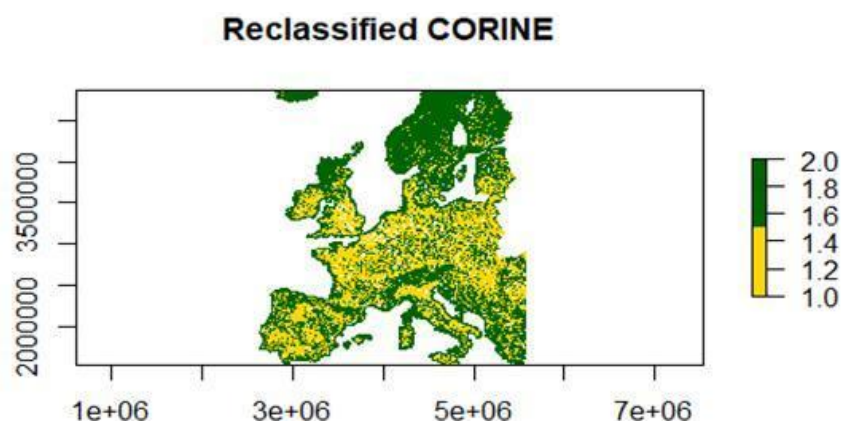


Figure 11: Reclassified Corine

2.3 Results

Spatial pattern interpretation

The model provides a clear visualization of geographic patterns in the vegetation dynamics across Europe. Of which we realised an evident north-south gradient in the vegetation stability and change.

In the northern and central regions, the landscape is mostly characterized by natural-stable syndrome (21) and natural-increasing syndrome (22). The pattern reflects stable and robust vegetation conditions, especially within forest ecosystems and semi-natural areas. This could be due to favorable climatic conditions or as well as conservation policies.

In southern Europe, we see a higher prevalence of agriculture-decreasing (10) and natural decreasing (20) patterns. These syndromes indicate areas exhibiting significant vegetation stress or disturbances and decline, probably due to a combination of various factors, e.g, climate change or agriculture abandonment, etc.

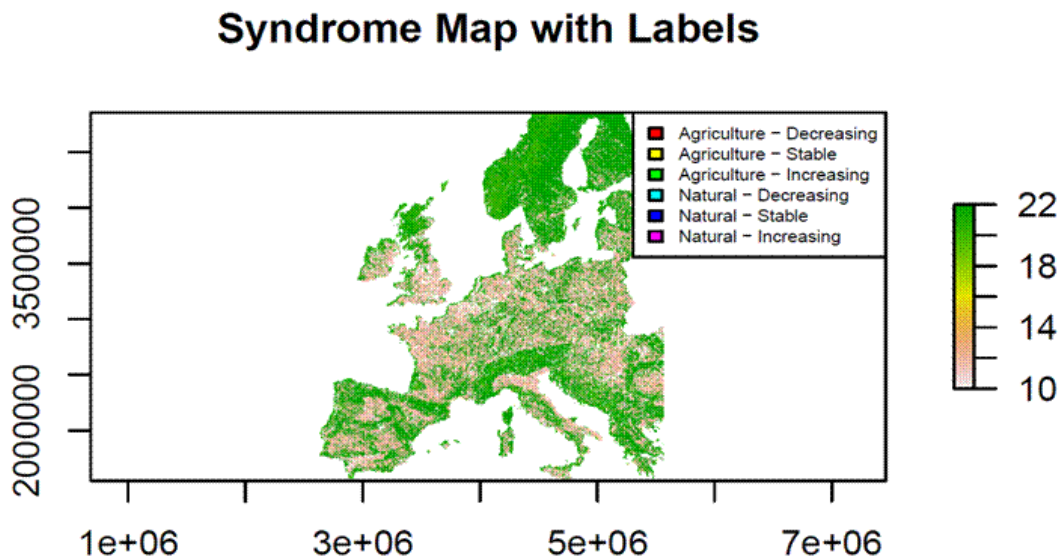


Figure 12: main syndrome map showing spatial distribution of the syndrome types

Here, the model identifies the difference between maximum and minimum growth seasons and calculates the mean.

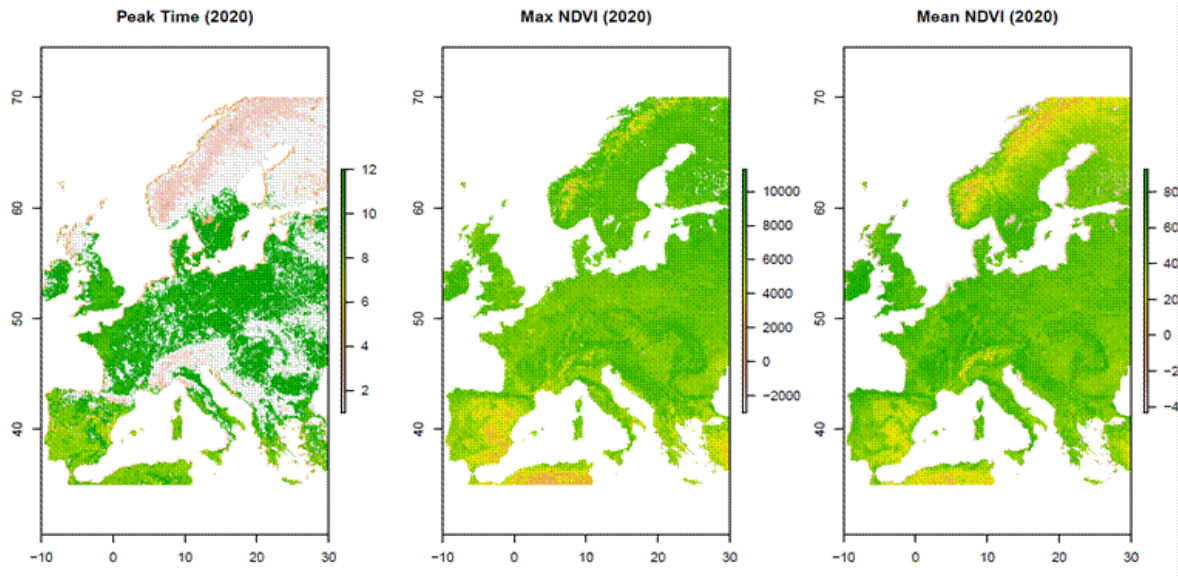


Figure 13: Phenological metrics of peak time, maximum NDVI, and Mean NDVI

Distribution of syndrome types

In our analysis, the results revealed distinct patterns in vegetation health and change across the study area. The dominant syndrome was predominantly those indicating healthy vegetation, to be specific, ‘Agriculture – Increasing’ (12) and ‘Natural – Increasing’ (22). This accounted for the highest pixel counts, revealing a highly productive and stable vegetation cover across several regions of Europe.

This was then followed by moderate syndromes highly influenced by decreasing irrigation, with moderate pixel counts of vegetation. This class of syndromes is ‘Agriculture–stable’ (11) and ‘Natural–stable’ (21), reflecting consistent vegetation as well as transitional areas like semi-natural areas.

The next syndrome was characterized by the lowest pixel count, which indicated declining vegetation health. ‘Agriculture-Decreasing’ (10) and ‘Natural-Decreasing’ (20). This class is characterized by low NDVI due to sparse or degraded vegetated areas. This could suggest stressed or disturbed areas. The table below shows the Quantitative distribution of the syndrome types across the study area.

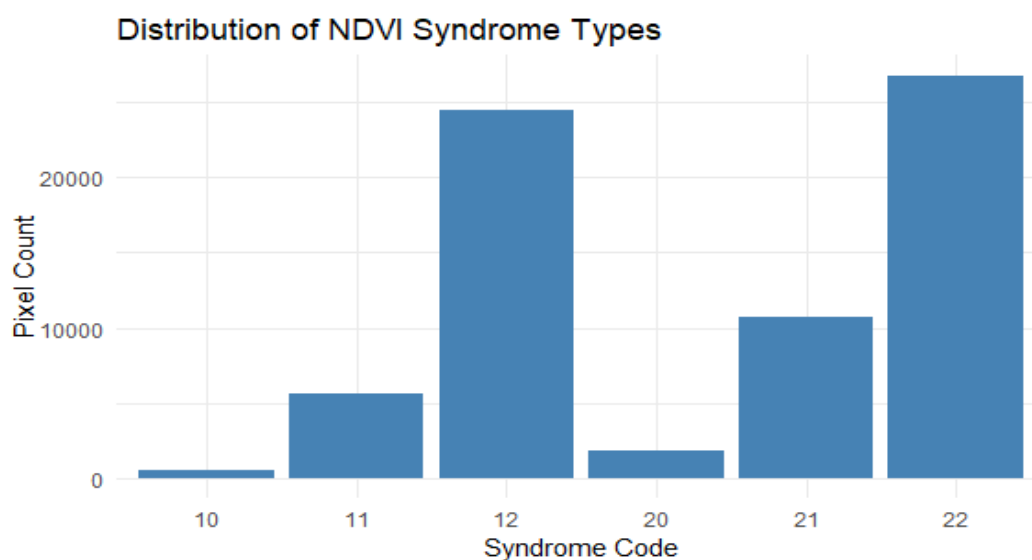
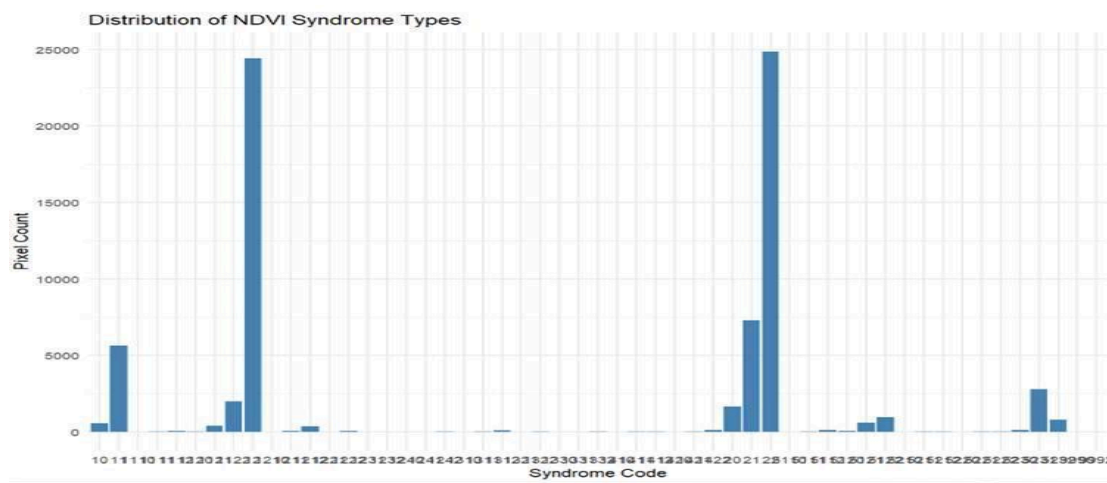


Figure 14: The quantitative Distribution of NDVI syndrome Types

Dominant Phenology Syndromes

The results of the analysis also showed combined phenological shifts patterns. The ‘Natural-NDVI Stable – Peak Earlier’ was the dominant syndrome, covering the largest geographical extent. This pattern suggests that while vegetation growth remains stable, the timing of the seasonal greening peak has shifted to an earlier period, perhaps due to climate change factors such as warmer seasons or altered rainfall patterns (Menzel et al., 2006).

Smaller but significant areas under ‘Agriculture – NDVI stable – Peak Earlier’ and ‘Natural -NDVI Decreasing -Peak Earlier’ show that both natural and agricultural systems are experiencing earlier phenological peaks; however, some natural areas show declining NDVI, thus hinting at emerging vegetation stress or disturbance.

The syndrome NDVI Decreasing – Peak stable coves very limited areas, indicating that there is a decline in vegetation health, though less frequently associated with unchanged peak

timing. while NA (representing non-vegetation or unclassified areas) for which metrics could not be substantially extracted, occupies the largest areas. The image in Figure (-) shows these findings.

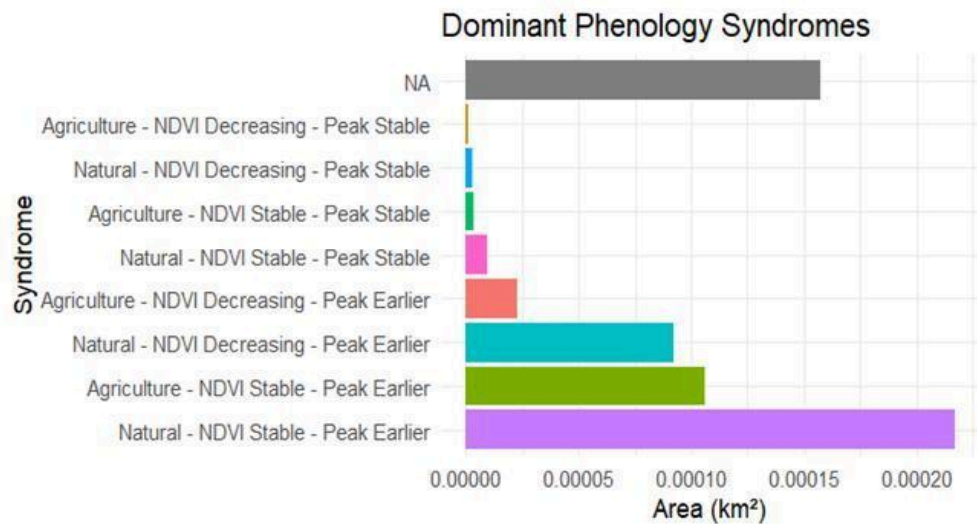


Figure 15: Dominant Phenology Syndromes

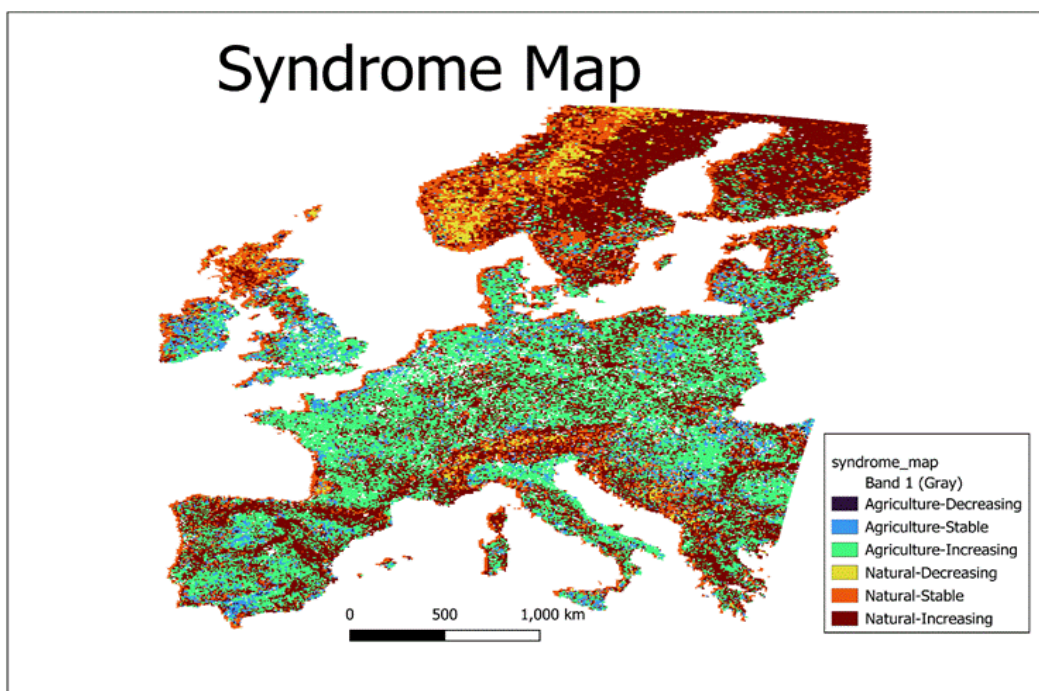


Figure 16: syndrome Map

2.4 Discussion and conclusion

Our analysis of the NDVI and Phenology syndromes across Europe from 1982 to 2022 shows a distinct spatial and temporal pattern in the vegetation dynamics. The syndrome mapping approach identified areas of stability, productivity, and decline in vegetation health alongside phenological shifts. In general, the increasing vegetation leads to an increase in productivity, which is seen in the regions of healthy, stable, and dense vegetation cover. The syndromes account for the highest proportion of the study area. The trend could be attributed to reforestation efforts, improved agricultural practices, natural regrowth, or effective conservation measures.

The moderate syndromes (11 and 21), which correspond to semi-natural areas, represent the transitional zones with vegetation conditions becoming moderate. While the syndromes 10 and 20 account for less than 2% of the NDVI trends, thus implying less or sparse vegetation or cover. These areas could be characterized by bare soils, degraded lands, or just zones affected by urbanization, thus signifying disturbances

The phenological analysis also showed that the dominant syndrome- ‘Natural - NDVI stable – Peak earlier’, indicates vegetation productivity being stable but with a shift towards earlier seasonal peaks. This trend could be due to climate changes, e.g, warmer temperatures and earlier onset of rainfall, which influences vegetation growth. The zones characterized by decreasing NDVI and earlier peaks may reflect regions where vegetation stress emerges due to disturbances.

All in all, our analyzed syndromes between 1982 to 2022 indicate that the landscape has experienced net improvement in vegetation health and productivity, with stable NDVI values being dominant across natural and agricultural classes. The shift towards earlier peaks confirms the influence of climatic changes on the vegetation pattern across the study period.

In conclusion, the syndrome approach is important for identifying, quantifying, and interpreting complex spatio-temporal patterns of vegetation change across Europe. By leveraging long-term satellite data, the analysis provides robust evidence of both ecological stability and adaptive results across Europe. The results offer a scientific base for sustainable land use planning, ecological and climate adaptation strategies, thus aiming to provide a balance between environmental conservation and productive land management across Europe.

Portfolio Part III: Implementation of a distributed lag model to analyze rainfall-biomass interactions.

3.1 Fit a distributed lag model (rainfall vs. NDVI) to the Botswana dataset.

$$\text{NDVI}_t = a_1 * \text{RF}_t + a_2 * \text{RF}_{t-1} + a_3 * \text{RF}_{t-2} + \dots + \text{RF}_{t-n}$$

Datasets

- Botswana_NDVI_USGS.tif: MODIS NDVI data
- rs_rain_chirps_Botswana.tif: CHIRPS rainfall data

This part of the portfolio focuses on the temporal relationship between vegetation dynamics and rainfall patterns using MODIS-derived NDVI and CHIRPS rainfall data over Botswana. The goal is to examine whether vegetation greenness (NDVI) can be explained not only by current rainfall but also by rainfall from previous months, thereby accounting for time-delayed ecological responses.

To begin, the NDVI and rainfall time series are decomposed using STL (Seasonal-Trend decomposition using Loess) to extract the residual (anomaly) components. These anomalies are then used to build distributed lag models (DLMs), which regress NDVI anomalies against rainfall anomalies from the same time step and previous lags (e.g., 1-month, 2-months, etc.). The Durbin-Watson test produced a DW value of 1.0944 and a p-value of 3.624e-10. While the DW statistic is below 2, indicating potential positive autocorrelation, the low p-value shows that this correlation is statistically significant.

Temporal Decomposition of NDVI and Rainfall Time Series

To investigate seasonal and trend components in the NDVI and rainfall data over Botswana (2000–2016), we performed a seasonal-trend decomposition using R's built-in `stl()` function (Seasonal and Trend decomposition using Loess).

- Figure 17: shows the decomposition of NDVI time series
- Figure 18: shows the decomposition of Rainfall time series

Each figure contains four panels:

- Data: The raw time series.
- Seasonal: Seasonal fluctuations, repeated annually.
- Trend: Long-term gradual increase or decrease.
- Remainder: The residuals (random noise and anomalies).

This decomposition helps isolate the true underlying trend and seasonality, which are crucial for modelling interactions between vegetation growth and rainfall.

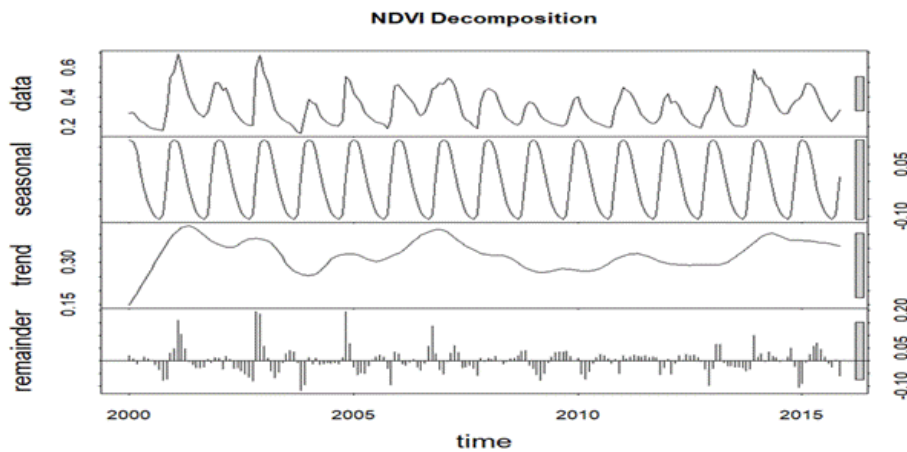


Figure 17: Temporal Decomposition of NDVI Data

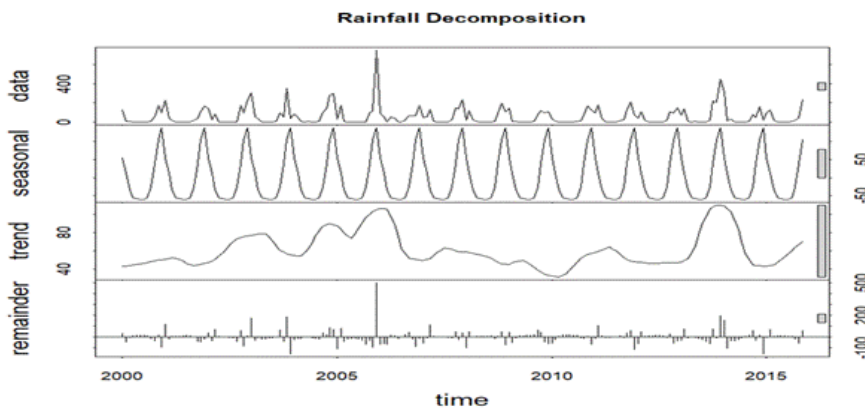


Figure 18: Temporal Decomposition of Rainfall Data.

3.2 Regression Analysis with Lagged Rainfall Variables

In order to examine the relationship between rainfall and NDVI, a series of linear regression models were created using different time lags of rainfall data. This approach helps to capture the delayed response of vegetation to rainfall, which is ecologically meaningful in semi-arid regions like Botswana.

Three models were developed:

- Figure.19: Model without lag.

This model uses the original rainfall data (no time shift) to predict NDVI. The residuals (left plot) still show a clear seasonal pattern, indicating that not all of the time-related variance in NDVI is explained by rainfall. The right plot shows the NDVI time series used in the model.

- Figure.20: Model with 1 month lag.

In this version, the rainfall data was shifted backward by one month ($\text{lag} = -1$), assuming that vegetation may respond to rainfall with a slight delay. While the residuals show slight

improvement in reducing seasonality compared to the no-lag model, clear autocorrelation patterns remain.

- **Figure 21: Model with 2-month lag**

This model uses rainfall values lagged by 0, 1, and 2 months. It considers cumulative effects and short-term memory in vegetation response. The residuals (left) appear somewhat flatter compared to previous models, indicating a better fit. However, the model still does not fully remove temporal autocorrelation.

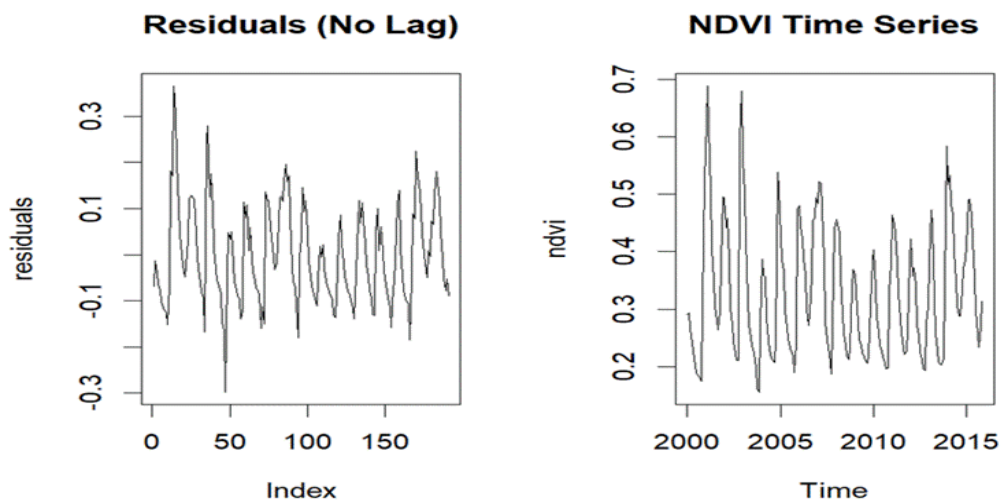


Figure 19: Model without lag.

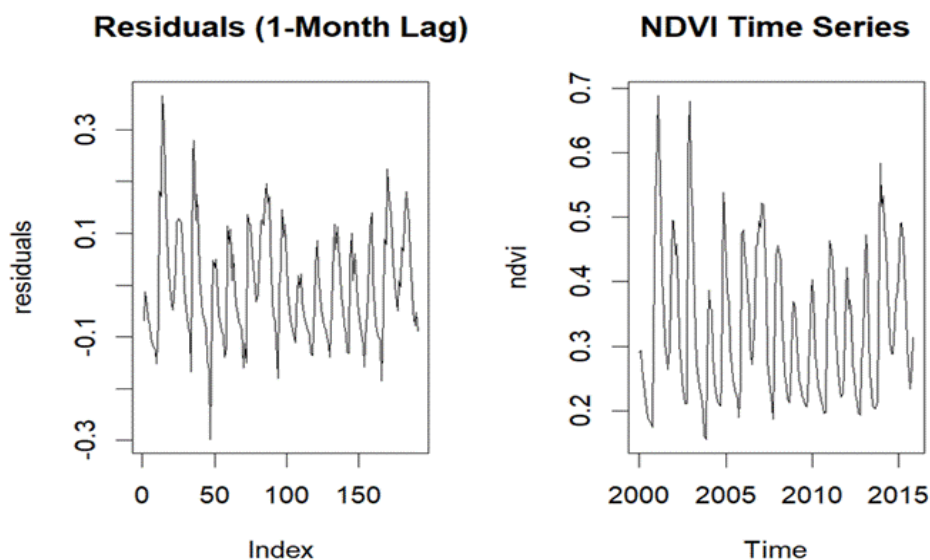


Figure 20: Model with 1 month lag.

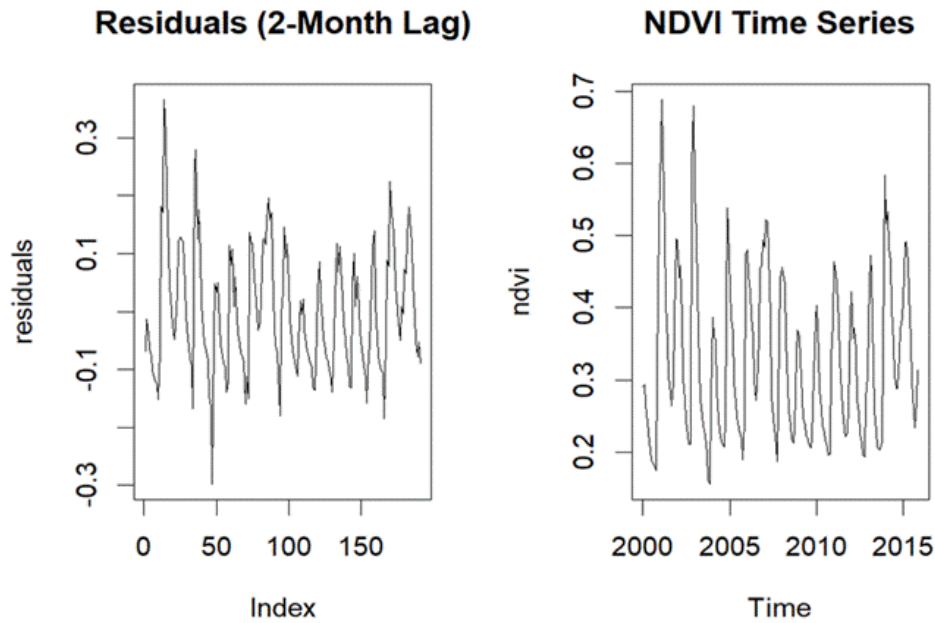


Figure 21: Model with 2-month lag.

Figures 19, 20, and 21 represent the residual patterns of linear regression models fitted between NDVI and rainfall data at different lag intervals. These plots help visualize how well each model captures the variability in NDVI after accounting for rainfall influences.

Upon applying a regression using the raw NDVI and rainfall data (no anomalies), the model yielded a correlation of 0.261, an R^2 value of 0.1256, and a highly significant p-value of $4.906e-07$, indicating a statistically meaningful relationship between the variables. However, seasonal trends remained evident in the residuals.

To assess the presence of autocorrelation in the residuals, a Durbin-Watson test was conducted, resulting in a DW statistic of 1.49417. This value suggests the presence of some positive autocorrelation, although it is closer to 2 than in previous models, indicating improved independence of residuals compared to models with no lag or fewer lag terms.

3.3 Model Diagnostics Based on NDVI Anomalies

The residual plots in these figures support the numerical results, highlighting areas where the model fails to fully capture NDVI variability, particularly during seasonal shifts.

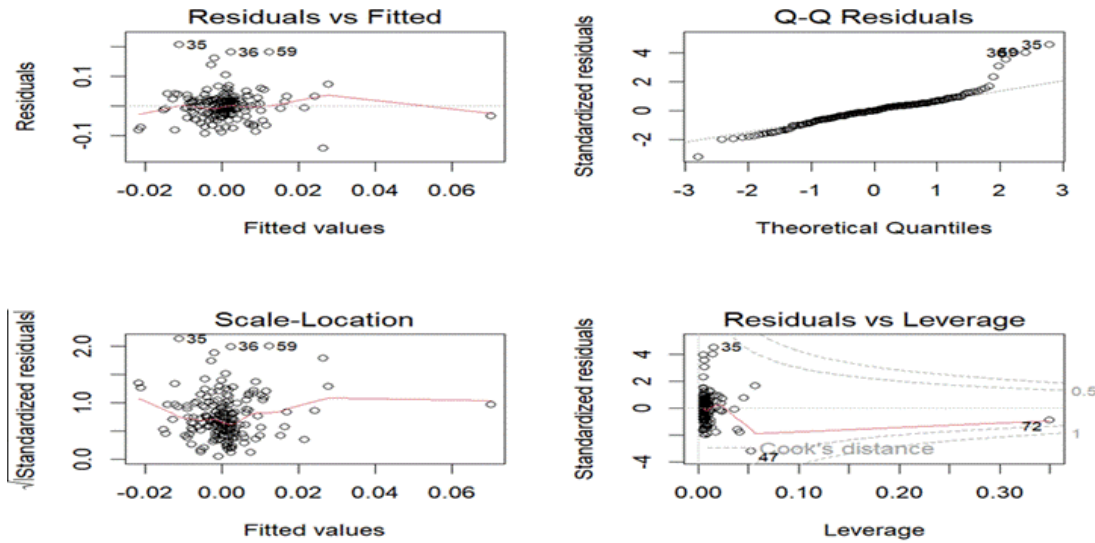


Figure 22: Model outputs using anomalies.

The distributed lag model (DLM) was developed to examine how vegetation greenness (NDVI anomalies) responds to rainfall anomalies over Botswana. The regression achieved an R^2 value of 0.198, meaning that about 20 % of NDVI variation is explained by rainfall, showing a moderate relationship between vegetation activity and precipitation patterns.

Among the rainfall lags included in the model, both lag 0 (current month rainfall, $p = 0.0286$) and lag 1 (previous month rainfall, $p = 0.0145$) were statistically significant. This indicates that vegetation reacts not only to immediate rainfall but also to rainfall from the preceding month, reflecting the short-term moisture storage and delayed growth response of plants.

The Durbin–Watson test ($DW = 1.9247$, $p = 0.3279$) shows no significant serial autocorrelation in the residuals, confirming that temporal dependence was largely reduced after applying the lag structure. However, since the model explains only part of NDVI variability, additional environmental factors such as soil moisture, temperature, and evapotranspiration may further improve the predictive accuracy.

3.4 Model Diagnostics for Distributed Lag Model (DLM)

These plots illustrate diagnostic checks for the final Distributed Lag Model (DLM) applied to NDVI–rainfall anomalies. The residuals appear randomly scattered around zero, and the Q–Q plot shows points close to the diagonal line, suggesting that model residuals are approximately normally distributed with minimal heteroscedasticity.

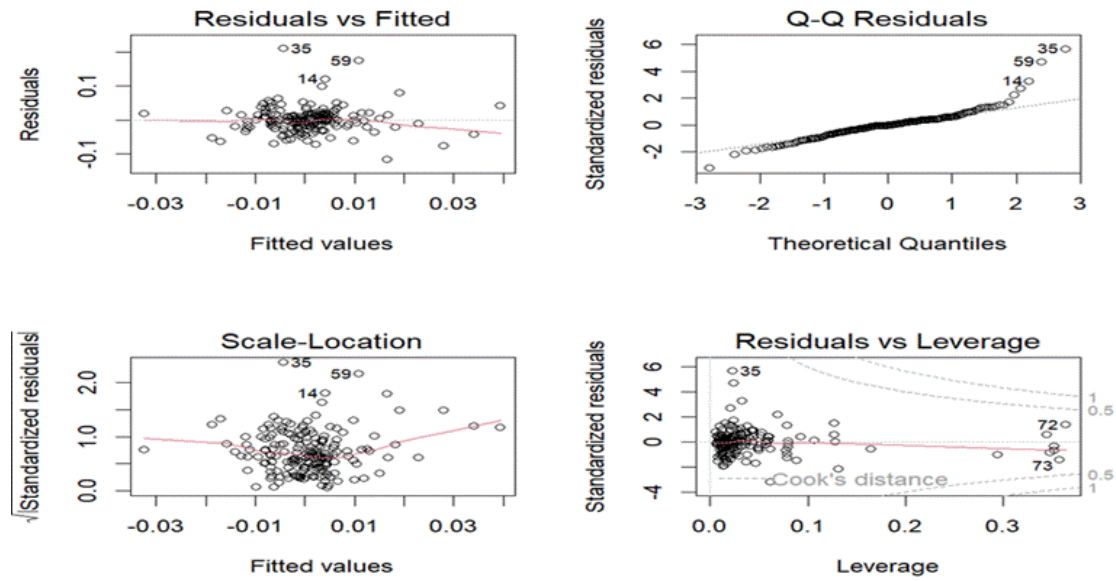


Figure 23: Model Graphs for Distributed lag models.

Compared to the earlier residual plots (Figure 22), these graphs indicate an improvement in model performance after introducing multiple lag terms. The residuals are more evenly dispersed, autocorrelation effects have reduced, and the model better accounts for delayed rainfall influence on NDVI.

3.5. Perform a regression analysis to find statistical relationships of the multiple F-values from the DLM and possible explanatory variables

To explore the statistical relationships between the F-values derived from the Distributed Lag Model (DLM) and various explanatory variables, a regression analysis was conducted for the Botswana region.

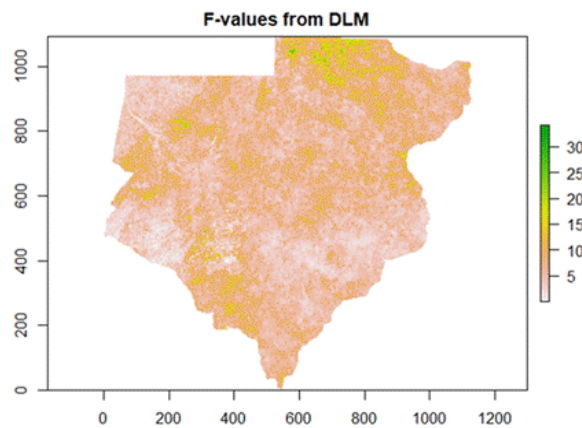


Figure 24: F-values derived from DLM

Using the terrain function from the raster library along with the Digital Elevation Model (DEM), several topographic indicators were extracted — including slope, aspect, flow direction, terrain roughness index (TRI), and topographic position index (TPI).

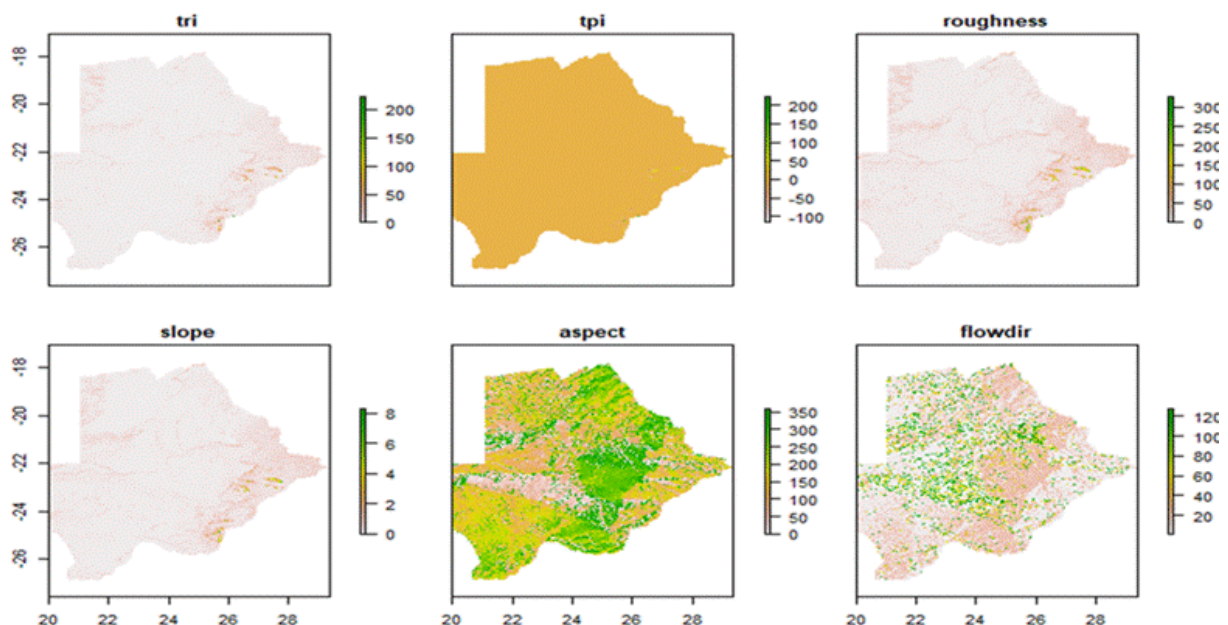


Figure 25: Spatial distribution of topographic variables across Botswana

Botswana region with different explanatory variables

The figure illustrates spatial variations of key topographic parameters derived from the Digital Elevation Model (DEM) using the terrain function. These include slope, aspect, flow direction, terrain roughness index (TRI), and topographic position index (TPI). Each variable provides insights into the landscape structure of Botswana - for instance, areas with high slope and roughness indicate elevated or rugged terrains, while variations in aspect and flow direction highlight potential differences in sunlight exposure and surface runoff. These layers

serve as explanatory variables in understanding the relationship between topography and vegetation response within the study area.

These variables were further combined with other environmental layers such as the topographic wetness index, vertical distance to the channel network (ACHAN), mean aridity, mean evapotranspiration, and mean surface temperature to form a comprehensive explanatory dataset.

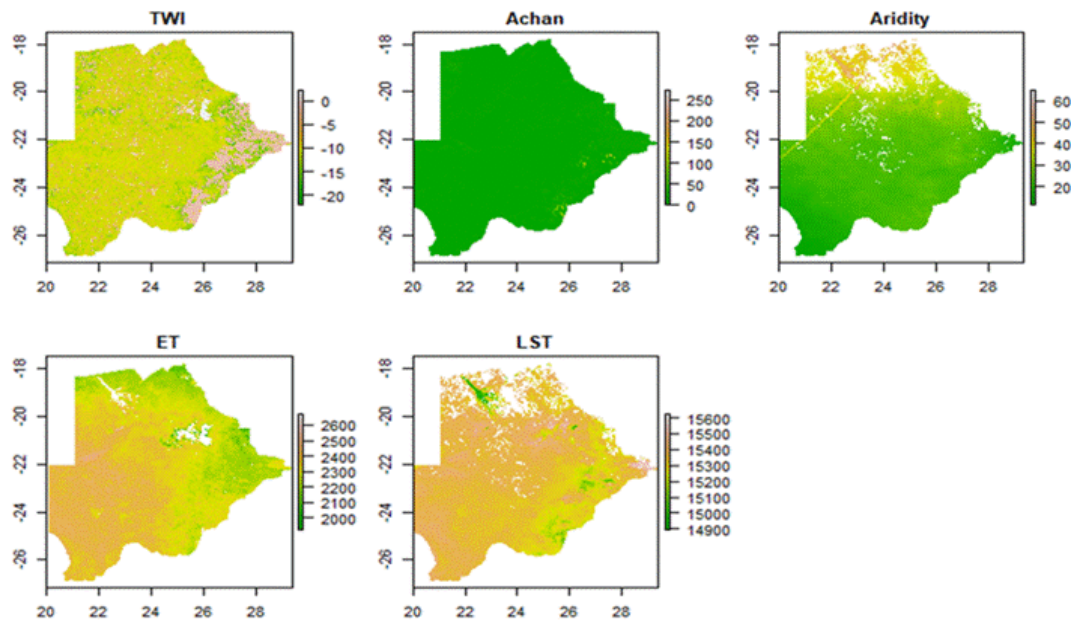


Figure 26: Environmental Variables.

Subsequently, a Random Forest regression model was trained on these spatial datasets, using the observed F-values as the dependent variable. The model's predictive F-values were compared with the observed ones. To address data imbalance between significant and non-significant pixels, a subset of 5,000 pixels (instead of 50,000) was randomly selected for both training and validation. This adjustment helped reduce computational load, as running the model with the full dataset often led to system crashes.

According to the variable importance results, aridity, aspect, land surface temperature (LST), and elevation together explained over 75% of the model's predictive power, indicating that these factors play a dominant role in influencing vegetation response. Conversely, parameters like surface roughness, ACHAN, and TRI exhibited relatively lower importance, contributing only up to 50%, suggesting a weaker influence on NDVI variability in the study region.

Relationship between Observed and Predicted F-values

The scatter plot (fig.27) displays the correlation between observed (test F-values) and predicted F-values derived from the Random Forest model. Each point represents a spatial pixel in the Botswana region. The moderate clustering pattern along the central diagonal indicates that the model can reasonably capture the relationship between rainfall-biomass interactions and explanatory variables such as slope, aspect, and elevation.

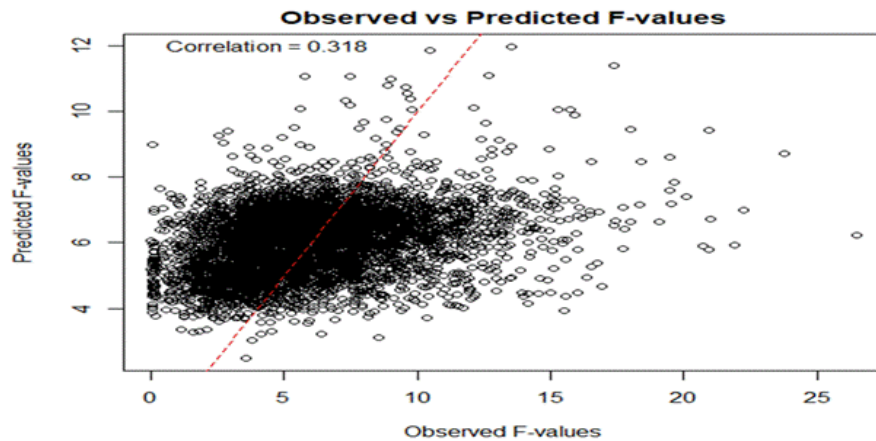


Figure 27: Correlation of F-values (Predicted v/s Observed)

Variable Importance in the Random Forest Model

The Random Forest regression model was used to assess the relative importance of different environmental variables influencing NDVI across Botswana. The results indicate that aridity, evapotranspiration, elevation and land surface temperature (LST) are the most influential factors, together contributing to over 75% of the model's predictive strength.

These parameters represent key climatic and topographic drivers that govern vegetation productivity in semi-arid regions like Botswana. Aridity directly affects water availability for plants, whereas elevation and aspect determine sunlight exposure and microclimatic variations, both of which impact vegetation growth patterns.

In contrast, variables such as slope, terrain roughness index (TRI), topographic wetness index (TWI), flow direction, and ACHAN (distance from the channel network) exhibit lower levels of importance, contributing less than 50% to the model's variance explanation. This suggests that while these terrain-related parameters influence local-scale hydrology and soil conditions, they play a secondary role compared to broader climatic controls. Overall, the model highlights that vegetation dynamics in Botswana are more strongly driven by large-scale environmental gradients—particularly climate and temperature—rather than by small-scale topographic features.

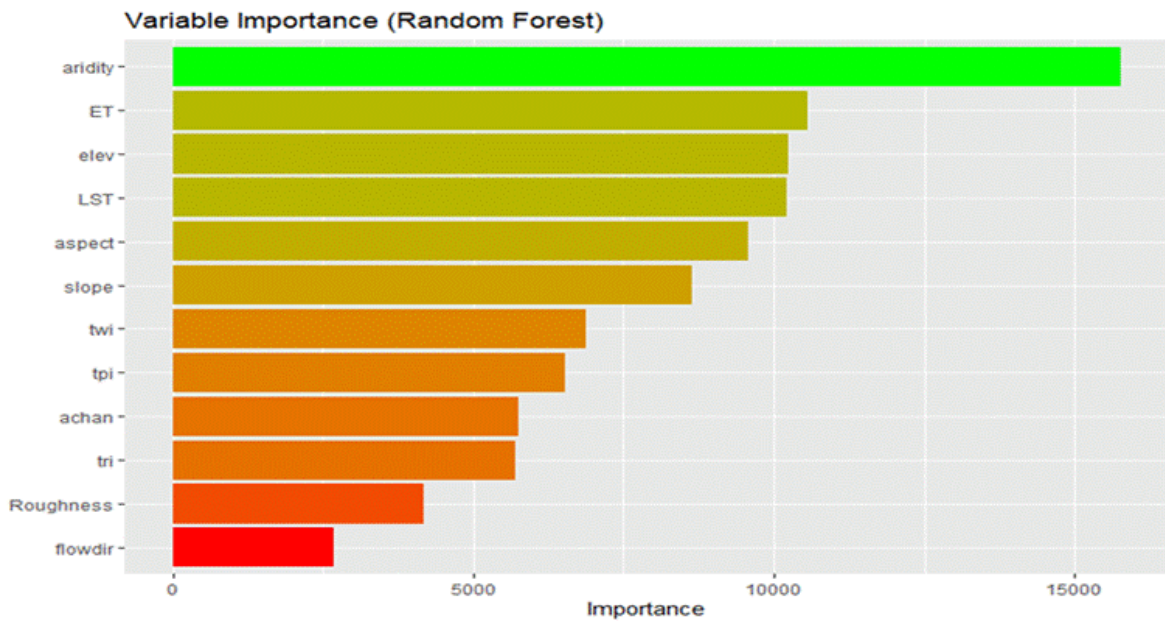


Figure 28: Graph showing variable of importance.

3.6 Results

The Distributed Lag Model (DLM) results revealed that rainfall significantly influences vegetation greenness across Botswana, with notable short-term lags. Both the current month rainfall (lag 0) and previous month rainfall (lag 1) were statistically significant ($p = 0.0286$ and $p = 0.0145$, respectively), indicating that vegetation responds not only to immediate rainfall but also to the moisture retained from preceding months. The overall model explained approximately 19.8% ($R^2 = 0.198$) of NDVI variation, suggesting that while rainfall is a primary driver of vegetation dynamics, other environmental factors also contribute to NDVI variability.

The model diagnostics confirmed that serial autocorrelation was minimized (Durbin–Watson = 1.9247, $p = 0.3279$), and residuals were approximately normally distributed. This demonstrates an improved model performance after including lag terms, emphasizing the relevance of temporal delays in semi-arid vegetation response to precipitation.

Further, the Random Forest regression analysis of F-values derived from the DLM identified aridity, elevation, land surface temperature (LST), and evapotranspiration as the most influential predictors, collectively explaining over 75% of the model’s predictive strength. These results underline that vegetation greenness in Botswana is strongly regulated by climatic constraints and topographic variability rather than by micro-terrain features such as slope or surface roughness.

Overall, the findings highlight that rainfall variability and environmental gradients together govern vegetation productivity in semi-arid ecosystems. The integration of DLM and Random Forest analysis successfully quantified both temporal and spatial patterns in rainfall–biomass interactions across Botswana.

3.7 Discussion and conclusion

In the third part of the portfolio, we investigated the rainfall–biomass relationship in Botswana using MODIS NDVI and CHIRPS rainfall data (2000–2016). The main goal was to understand the delayed vegetation response to rainfall through the implementation of a Distributed Lag Model (DLM) and complementary regression analyses.

We began by decomposing the NDVI and rainfall time series using Seasonal-Trend decomposition (STL) to separate long-term trends, seasonal cycles, and residual anomalies. These anomalies were then used to build DLMs to capture the influence of both current and lagged rainfall on NDVI anomalies. Results showed that rainfall from both the current month (lag 0) and the previous month (lag 1) significantly influenced vegetation growth ($p < 0.05$). The model explained about 20% ($R^2 = 0.198$) of the NDVI variability, highlighting the short-term but important influence of precipitation on vegetation greenness in semi-arid regions.

To further explain spatial variations in the vegetation–rainfall interaction, we analyzed F-values derived from the DLM against multiple environmental and topographic variables using a Random Forest regression model. Key variables such as aridity, elevation, land surface temperature (LST), and evapotranspiration were found to be the dominant factors controlling vegetation response, together explaining more than 75% of the predictive power.

Overall, Portfolio 3 demonstrated the usefulness of distributed lag modelling for capturing time-delayed ecological processes in semi-arid systems. It also emphasized the combined influence of climatic and topographic drivers on vegetation dynamics, contributing to a more comprehensive understanding of rainfall–NDVI interactions over Botswana.

6. References

- 1 Ya Liu, Yan Li, Shuangcheng Li, Safa Motesharrei, Spatial and Temporal Patterns of Global NDVI Trends: Correlations with Climate and Human Factors, *Remote Sens.* 2015, 7(10), 13233–13250. URL: <https://www.mdpi.com/2072-4292/7/10/13233>
- 2 Pettorelli, N., Vik, J. O., Mysterud, A., Gaillard, J.-M., Tucker, C. J., & Stenseth, N. C. (2005). Using the satellite-derived NDVI to assess ecological responses to environmental change. *Trends in Ecology & Evolution*, 20(9), 503–510. <https://doi.org/10.1016/j.tree.2005.05.011>
- 3 Liu, Y., Li, Y., Li, S., & Motesharrei, S. (2015). Spatial and temporal patterns of global NDVI trends: Correlations with climate and human factors. *Remote Sensing*, 7(10), 13233–13250. <https://doi.org/10.3390/rs71013233>
- 4 Eisfelder, C., Asam, S., Hirner, A., Reiners, P., Holzwarth, S., Bachmann, M., Gessner, U., Dietz, A., Huth, J., Bachofer, F., & Kuenzer, C. (2023). Seasonal vegetation trends for Europe over 30 years from a novel Normalised Difference Vegetation Index (NDVI) time-series—The TIMELINE NDVI product. *Remote Sensing*, 15(15), 3616. <https://doi.org/10.3390/rs15153616>
- 5 Belsley, D. A., Kuh, E., & Welsch, R. E. (2005). *Regression Diagnostics: Identifying Influential Data and Sources of Collinearity*. Wiley-Interscience. <https://doi.org/10.1002/0471725153>
- 6 Fox, J. (2015). *Applied Regression Analysis and Generalized Linear Models* (3rd ed.). SAGE Publications. <https://us.sagepub.com/en-us/nam/applied-regression-analysis-and-generalized-linear-models/book241633>
- 7 Chatterjee, S., & Hadi, A. S. (2015). *Regression Analysis by Example* (5th ed.). Wiley. <https://doi.org/10.1002/9781119122733>
- 8 Faraway, J. J. (2014). *Linear Models with R* (2nd ed.). CRC Press. <https://doi.org/10.1201/b17144>
- 9 Fox, J., & Weisberg, S. (2019). *An R Companion to Applied Regression* (3rd ed.). SAGE Publications. <https://socialsciences.mcmaster.ca/jfox/Books/Companion/>
- 10 Gelman, A., & Hill, J. (2007). *Data Analysis Using Regression and Multilevel/Hierarchical Models*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511790942>
- 11 James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning: With Applications in R* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-0716-1418-1>
- 12 Menzel, A., Sparks, T. H., Estrella, N., Koch, E., Aasa, A., Ahas, R., Alm-Kübler, K., Bissolli, P., Braslavská, O., Briede, A., Chmielewski, F. M., Crepinsek, Z., Curnel, Y., Dahl, Å., Defila, C., Donnelly, A., Filella, Y., Jatczak, K., Måge, F., ... Zust, A. (2006). European phenological response to climate change matches the warming pattern. *Global Change Biology*, 12(10), 1969–1976. <https://doi.org/10.1111/j.1365-2486.2006.01193.x>
- 13 Stellmes, M., Röder, A., Udelhoven, T., & Hill, J. (2013). Mapping syndromes of land change in Spain with remote sensing time series, demographic and climatic data. *Land Use Policy*, 30(1), 685–702. <https://doi.org/10.1016/j.landusepol.2012.05.007>

7. Appendix

R code for Portfolio 1

```
# Install and load required libraries
install.packages(c("raster", "rgdal", "Kendall", "trend", "rasterVis", "RColorBrewer"), dependencies = TRUE)
setwd("D:/Pattern_Recognition/Portfolios/Portfolio_1")
# 1. Load NDVI raster (GIMMS monthly 1982–2022)
filename <- "D:/Pattern_Recognition/Portfolios/Portfolio_1/GIMMS_monthly_1982-2022.tif"
ndvi_raster <- brick(filename)
# Check basic raster info before proceeding
print(ndvi_raster) # See raster info
nlayers(ndvi_raster) # Should return 492 layers (41 years x 12 months)
# Plot first year (1982)
ndvi_1982 <- subset(ndvi_raster, 1:12)
plot(ndvi_1982, col = rev(terrain.colors(10)), legend = TRUE, main = "NDVI 1982 (Monthly)")
# 2. Data Manipulation (Convert NDVI raster to array for analysis)
cube <- as.array(ndvi_raster)
# cube is a 3D array (rows, cols, time). #nrow() and ncol() typically work on matrices but for arrays, we want dim(cube) which returns all
dimensions. So, we used:
dims <- dim(cube) # dims[1] = rows, dims[2] = cols, dims[3] = time layers
nrows <- dims[1]
ncols <- dims[2]
ntimes <- dims[3]
print(dim(cube)) # Check dimensions and range before scaling
print(range(cube, na.rm = TRUE))
cube <- cube / 10000 # GIMMS NDVI is usually scaled 0–10000
print(range(cube, na.rm = TRUE)) # Inspect range after scaling
cube[cube < -0.1 | cube > 1] <- NA # Remove invalid NDVI values (< -0.1 or > 1)
cube[cube == -9999] <- NA # Remove known no-data values
print(range(cube, na.rm = TRUE)) # Final check after cleaning
# Prepare empty result arrays
tau_array <- array(NA, c(nrows, ncols))
pval_array <- array(NA, c(nrows, ncols))
regcoeff <- array(NA, c(nrows, ncols))
# 3. Trend Analysis
# Loop through all pixels and apply Seasonal Mann-Kendall
library(zoo) # for na.approx interpolation
library(trend) # for SeasonalMannKendall
library(Kendall) # for MannKendall fallback
pb <- winProgressBar(title = "Trend Analysis Progress", min = 0, max = nrows, width = 300)
start.time <- Sys.time()
for (i in 1:nrows) {
  setWinProgressBar(pb, i, title = paste(round(i / nrows * 100, 0), "% done"))
  for (j in 1:ncols) {
    x <- cube[i, j, ] # extract time-series for pixel

    # Skip if all NA or too few non-NA points
    if (all(is.na(x)) || sum(!is.na(x)) < 24) {
      tau_array[i, j] <- NA
      pval_array[i, j] <- NA
      regcoeff[i, j] <- NA
      next
    }

    # Interpolate missing values (linear interpolation)
    x_filled <- tryCatch({
      na.approx(x, na.rm = FALSE)
    }, error = function(e) {
      rep(NA, length(x)) # if interpolation fails, return NA vector
    })

    # If interpolation failed or edge NAs remain, skip
    if (any(is.na(x_filled))) {
      tau_array[i, j] <- NA
      pval_array[i, j] <- NA
      regcoeff[i, j] <- NA
      next
    }

    # Check variance > 0 after interpolation
```

```

    if (var(x_filled) <= 0) {
      tau_array[i, j] <- NA
      pval_array[i, j] <- NA
      regcoeff[i, j] <- NA
      next
    }

    # Convert to time series with monthly frequency starting 1982 Jan
    x_ts <- ts(x_filled, start = c(1982, 1), frequency = 12)

    # Try Seasonal Mann-Kendall test
    res <- tryCatch({
      suppressWarnings(SeasonalMannKendall(x_ts))
    }, error = function(e) NULL,
      warning = function(w) NULL)

    # If Seasonal Mann-Kendall fails, fallback to Mann-Kendall
    if (is.null(res)) {
      res <- tryCatch({
        MannKendall(x_filled)
      }, error = function(e) NULL,
        warning = function(w) NULL)

      if (is.null(res)) {
        tau_array[i, j] <- NA
        pval_array[i, j] <- NA
      } else {
        tau_array[i, j] <- res$tau
        pval_array[i, j] <- res$sl
      }
    } else {
      tau_array[i, j] <- res$tau
      pval_array[i, j] <- res$sl * as.numeric(sign(res$tau))
    }

    # Linear regression slope on filled data
    time <- 1:length(x_filled)
    model <- lm(x_filled ~ time)
    regcoeff[i, j] <- model$coefficients[2]
  }
}
close(pb)
end.time <- Sys.time()
print(paste("Trend analysis completed in", round(difftime(end.time, start.time, units = "mins"), 2), "minutes"))

# 4. Convert results arrays back into raster format (full year results)
tau_raster <- raster(tau_array,
  xmn = extent(ndvi_raster)[1],
  xmx = extent(ndvi_raster)[2],
  ymn = extent(ndvi_raster)[3],
  ymx = extent(ndvi_raster)[4],
  crs = projection(ndvi_raster))
pval_raster <- raster(pval_array,
  xmn = extent(ndvi_raster)[1],
  xmx = extent(ndvi_raster)[2],
  ymn = extent(ndvi_raster)[3],
  ymx = extent(ndvi_raster)[4],
  crs = projection(ndvi_raster))
regcoeff_raster <- raster(regcoeff,
  xmn = extent(ndvi_raster)[1],
  xmx = extent(ndvi_raster)[2],
  ymn = extent(ndvi_raster)[3],
  ymx = extent(ndvi_raster)[4],
  crs = projection(ndvi_raster))

# 5. VISUALIZE RESULTS
# Set up side-by-side plots
par(mfrow = c(1, 3), mar = c(5, 4, 4, 6)) # Extra space for side legend
# 1. Tau Plot
plot(tau_raster,
  main = "Mann-Kendall Tau (NDVI Trend)",
  col = colorRampPalette(c("blue", "white", "red"))(100),
  zlim = c(-1, 1),
  legend.args = list(text = "Tau Coefficient", side = 4, font = 2, line = 2.5, cex = 0.8))
legend("bottomleft", legend = c("Red: Positive trend", "White: No trend", "Blue: Negative trend"),

```

```

      fill = c("red", "white", "blue"), cex = 0.8, bty = "n")
# 2. P-value Plot
plot(pval_raster,
      main = "P-value (Trend Significance)",
      col = colorRampPalette(c("darkgreen", "yellow", "white"))(100),
      zlim = c(0, 1),
      legend.args = list(text = "P-value", side = 4, font = 2, line = 2.5, cex = 0.8))
legend("bottomleft", legend = c("Dark green: Highly significant (p<0.05)",
                                "Yellow: Borderline", "White: Not significant"),
      fill = c("darkgreen", "yellow", "white"), cex = 0.8, bty = "n")
# 3. Sen's Slope Plot (Fixed zlim to show full color range)
# Calculate max absolute value to create symmetrical color scale
max_slope <- max(abs(values(regcoeff_raster)), na.rm = TRUE)
plot(regcoeff_raster,
      main = "Sen's Slope (NDVI Change/Year)",
      col = colorRampPalette(c("brown", "white", "darkgreen"))(100),
      zlim = c(-max_slope, max_slope), # Force symmetrical scale
      legend.args = list(text = "Slope Value", side = 4, font = 2, line = 2.5, cex = 0.8))
legend("bottomleft", legend = c("Brown: Strong decline", "White: Stable", "Dark green: Strong increase"),
      fill = c("brown", "white", "darkgreen"), cex = 0.8, bty = "n")

# Reset layout
par(mfrow = c(1, 1))
# 6. Save as GeoTIFF
out_dir <- "D:/Pattern_Recognition/Portfolios/Portfolio_1/outputs"
dir.create(out_dir, showWarnings = FALSE) # create if doesn't exist
writeRaster(tau_raster, filename = file.path(out_dir, "full_tau_raster.tif"), format = "GTiff", overwrite = TRUE)
writeRaster(pval_raster, filename = file.path(out_dir, "full_pval_raster.tif"), format = "GTiff", overwrite = TRUE)
writeRaster(regcoeff_raster, filename = file.path(out_dir, "full_regcoeff_raster.tif"), format = "GTiff", overwrite = TRUE)
Q.1 Is there evidence for the “greening trend” hypothesis in natural vegetation in relation to climate change using the full temporal period?
# 1. Load Required Libraries and data
lapply(c("raster", "terra", "zyp", "Kendall", "RColorBrewer", "tmap"), library, character.only = TRUE)
setwd("D:/Pattern_Recognition/Portfolios/Portfolio_1")
# Load GIMMS monthly NDVI data (1982–2022) as a raster stack
ndvi_raster <- stack("GIMMS_monthly_1982-2022.tif")
# 2. Preprocess NDVI Data
ndvi_raster_scaled <- ndvi_raster / 10000 # Rescale NDVI (GIMMS values typically scaled by 10,000)
range(ndvi_raster_scaled[, , na.rm = TRUE]) # Check the current range of scaled NDVI
ndvi_raster_scaled[ndvi_raster_scaled < -0.1 | ndvi_raster_scaled > 1] <- NA # Remove invalid values
ndvi_raster_scaled[ndvi_raster_scaled == -9999] <- NA # Optional: Remove specific no-data values (-9999)
range(ndvi_raster_scaled[, , na.rm = TRUE]) # Confirm cleaned value range
# 3. Compute Annual NDVI (Mean of 12 months)
years <- 1982:2022
annual_ndvi_list <- lapply(years, function(y) {
  start_month <- (y - 1982) * 12 + 1
  end_month <- start_month + 11
  mean(ndvi_raster_scaled[[start_month:end_month]], na.rm = TRUE)
})
annual_ndvi_stack <- stack(annual_ndvi_list)
names(annual_ndvi_stack) <- years
# Convert to terra format for processing
annual_ndvi_v <- rast(annual_ndvi_stack)
years_vec <- years
# 4. Compute Sen's Slope per Pixel (Trend)
sen_slope_raster <- app(annual_ndvi_v, function(x) {
  if (all(is.na(x))) return(NA)
  df <- data.frame(year = years_vec, value = x)
  zyp.sen(value ~ year, data = df)$coefficients[2]
})
# Classify trend: 1 = greening, -1 = browning, 0 = no trend
greening_browning <- app(sen_slope_raster, function(x) {
  ifelse(is.na(x), NA, ifelse(x > 0, 1, ifelse(x < 0, -1, 0)))
})
# 5. Kendall Tau Test (Significance)
tau_pval_raster <- app(annual_ndvi_v, function(x) {
  x <- na.omit(x)
  if (length(x) < 3 || var(x) == 0) return(NA)
  Kendall(x, years_vec[1:length(x)])$sl[1] # return p-value
})
# Mask to show only statistically significant trends (p < 0.05)
significant_trends <- ifel(
  is.na(greening_browning) | is.na(tau_pval_raster),
  NA,

```

```

    ifel(tau_pval_raster < 0.05, greening_browning, 0)
  )
# 6. Compute Absolute & Percent NDVI Change
years_period <- 41
absolute_change <- sen_slope_raster * years_period
# Get NDVI baseline from 1982
ndvi_1982 <- rast(annual_ndvi_stack[[1]])
percent_change <- app(c(absolute_change, ndvi_1982), function(x) {
  change <- x[1]
  ref <- x[2]
  if (is.na(ref) | ref == 0) return(NA)
  return((change / ref) * 100)
})
# Cap extreme percent change values to ±100%
percent_change_capped <- clamp(percent_change, lower = -100, upper = 100)
# 7. Summary Statistics
num_greening <- sum(values(sen_slope_raster) > 0, na.rm = TRUE)
num_browning <- sum(values(sen_slope_raster) < 0, na.rm = TRUE)
num_total <- sum(!is.na(values(sen_slope_raster)))
percent_greening <- num_greening / num_total * 100
percent_browning <- num_browning / num_total * 100
cat(sprintf("🌱 %.2f%% of area is greening.\n", percent_greening))
cat(sprintf("🌵 %.2f%% of area is browning.\n", percent_browning))
# Summary of percent change
summary(values(percent_change), na.rm = TRUE)
summary(values(percent_change_capped), na.rm = TRUE)
# 8. Visualizations
# Histogram of percent change
hist(values(percent_change_capped),
      breaks = 100,
      col = "darkgreen",
      main = "NDVI Percent Change (1982–2022)",
      xlab = "Percent Change (%)")
# Define diverging color palette
cols <- colorRampPalette(brewer.pal(11, "RdYlGn"))(100)
breaks <- seq(-100, 100, by = 20)
# Plot percent change
plot(percent_change_capped,
      col = cols,
      breaks = breaks,
      main = "NDVI Percent Change (1982–2022)",
      legend = TRUE,
      plg = list(title = "Change (%)", cex = 0.8))
# Plot trend classification
plot(greening_browning,
      col = brewer.pal(3, "RdYlGn"),
      main = "NDVI Trend: Greening / Browning")
# Plot only significant trends
plot(significant_trends,
      col = colorRampPalette(c("brown", "white", "green"))(100),
      main = "Significant NDVI Trends (p < 0.05)")
# Optional: Improved map using `tmap`
tm_shape(significant_trends) +
  tm_raster(palette = c("brown", "yellow", "green", "darkgreen"),
            title = "Trend",
            midpoint = 0,
            colorNA = "white") +
  tm_layout(main.title = "Significant NDVI Trends")
# 9. Export Output Rasters
writeRaster(sen_slope_raster, "NDVI_trend_sen_slope.tif", overwrite = TRUE)
writeRaster(greening_browning, "NDVI_trend_greening_browning.tif", overwrite = TRUE)
writeRaster(significant_trends, "NDVI_significant_trends.tif", overwrite = TRUE)
writeRaster(percent_change_capped, "percent_change_ndvi_1982_2022_capped.tif", overwrite = TRUE)
# Q.2. Are there changes in trend strengths, if only the summer months for each year are included in the analysis (April-September)?
# 1. Load Required Libraries and data
required_packages <- c("sp", "raster", "terra", "sf", "progress", "trend", "Kendall", "rasterVis", "RColorBrewer", "animation", "rkt", "zoo",
  "tmap")
new_pkgs <- required_packages[!(required_packages %in% installed.packages()[,"Package"])]
if(length(new_pkgs)) install.packages(new_pkgs)
invisible(lapply(required_packages, library, character.only = TRUE))
setwd("D:/Pattern_Recognition/Portfolios/Portfolio_1/outputs")
ndvi_raster <- brick("D:/Pattern_Recognition/Portfolios/Portfolio_1/GIMMS_monthly_1982-2022.tif")

```

2. Data Manipulation: Extract Only Summer Months (April to September)

```
# Create a list of layers corresponding to the summer months (April to September)
summer_months <- c(4, 5, 6, 7, 8, 9)
n_years <- nlayers(ndvi_raster) / 12
# Generate indices for all summer months across all years
summer_indices <- rep(summer_months, times = n_years) +
  rep((0:(n_years - 1)) * 12, each = length(summer_months))
summer_ndvi <- ndvi_raster[[summer_indices]] # Extract summer NDVI for all years
print(summer_ndvi) # Verify extraction
nlayers(summer_ndvi) # Should be n_years * 6 months per year
```

3. Trend Analysis for Summer Months Only

```
library(zoo) # for na.approx
library(rkt) # for rkt()
library(raster)
# Convert NDVI raster stack to 3D array for April–September (already extracted earlier)
summer_cube <- as.array(summer_ndvi)
# Get dimensions
nrows <- dim(summer_cube)[1]
ncols <- dim(summer_cube)[2]
ntimes <- dim(summer_cube)[3] # should be years * 6
# Initialize arrays for results
tau_summer <- array(NA, c(nrows, ncols))
pval_summer <- array(NA, c(nrows, ncols))
regcoeff_summer <- array(NA, c(nrows, ncols))
# Define proper time and block vectors
start_year <- 1982
n_years <- ntimes / 6
# Create time vector: e.g., 1982.25 for April, 1982.33 for May, etc.
months_fraction <- (4:9) / 12 # April (4) to September (9)
years <- rep(start_year:(start_year + n_years - 1), each = 6)
time_vec <- years + rep(months_fraction, times = n_years)
# Create block vector: one block per year (6 months each)
block_summer <- rep(1:n_years, each = 6)
```

#4. Loop over each pixel

```
pb_summer <- winProgressBar(title = "Summer Trend Analysis Progress", min = 0, max = nrows, width = 300)
start.time_summer <- Sys.time()
for (i in 1:nrows) {
  setWinProgressBar(pb_summer, i, title = paste(round(i / nrows * 100, 0), "% done"))
  for (j in 1:ncols) {
    x <- summer_cube[i, j, ] # Time series for pixel

    # Skip if all NA or too short
    if (all(is.na(x)) || sum(!is.na(x)) < 6) {
      next
    }
    # Interpolate missing values
    x_filled <- tryCatch({
      na.approx(x, na.rm = FALSE)
    }, error = function(e) rep(NA, length(x)))

    if (any(is.na(x_filled))) {
      next
    }

    # Modified Seasonal Mann-Kendall test
    res_summer <- tryCatch({
      rkt_res <- rkt(time_vec, x_filled, block = block_summer, rep = "mean")
      list(tau = rkt_res$tau, pval = rkt_res$pval)
    }, error = function(e) NULL)
    if (!is.null(res_summer)) {
      tau_summer[i, j] <- res_summer$tau
      pval_summer[i, j] <- res_summer$pval
    }
    # Linear regression slope
    model_summer <- lm(x_filled ~ time_vec)
    regcoeff_summer[i, j] <- model_summer$coefficients[2]
  }
}
close(pb_summer)
end.time_summer <- Sys.time()
print(paste("Summer trend analysis completed in", round(difftime(end.time_summer, start.time_summer, units = "mins"), 2), "minutes"))
```

5. Convert Results Arrays Back into Raster Format

```
# Convert your results arrays (tau_summer, pval_summer, regcoeff_summer) back into raster format
summer_tau_raster <- raster(tau_summer,
```

```

        xmn = extent(ndvi_raster)[1],
        xmx = extent(ndvi_raster)[2],
        ymn = extent(ndvi_raster)[3],
        ymx = extent(ndvi_raster)[4],
        crs = projection(ndvi_raster))
summer_pval_raster <- raster(pval_summer,
        xmn = extent(ndvi_raster)[1],
        xmx = extent(ndvi_raster)[2],
        ymn = extent(ndvi_raster)[3],
        ymx = extent(ndvi_raster)[4],
        crs = projection(ndvi_raster))
summer_regcoeff_raster <- raster(regcoeff_summer,
        xmn = extent(ndvi_raster)[1],
        xmx = extent(ndvi_raster)[2],
        ymn = extent(ndvi_raster)[3],
        ymx = extent(ndvi_raster)[4],
        crs = projection(ndvi_raster))

# 6. Plotting
# Extract actual ranges from data
tau_range <- range(values(summer_tau_raster), na.rm = TRUE)
pval_range <- range(values(summer_pval_raster), na.rm = TRUE)
slope_range <- range(values(summer_regcoeff_raster), na.rm = TRUE)
max_slope_abs <- max(abs(slope_range)) # Use symmetric range for slope
slope_zlim <- c(-max_slope_abs, max_slope_abs)
# Set up 3 plots side by side
par(mfrow = c(1, 3), mar = c(5, 4, 4, 6))
# 1. Tau Plot
plot(summer_tau_raster,
      main = "Summer NDVI: Mann-Kendall Tau (Apr-Sep)",
      col = colorRampPalette(c("darkorchid4", "white", "darkorange3"))(100),
      zlim = tau_range,
      legend.args = list(text = "Tau Coefficient", side = 4, font = 2, line = 2.5, cex = 0.8))
legend("bottomleft", legend = c("Dark orange: Positive trend", "White: No trend", "Dark purple: Negative trend"),
      fill = c("darkorange3", "white", "darkorchid4"), cex = 0.8, bty = "n")
# 2. P-value Plot
plot(summer_pval_raster,
      main = "Summer NDVI: P-value (Apr-Sep)",
      col = colorRampPalette(c("Cyan", "deepskyblue3", "white"))(100),
      zlim = c(0, 1),
      legend.args = list(text = "P-value", side = 4, font = 2, line = 2.5, cex = 0.8))
legend("bottomleft", legend = c("Cyan: Highly significant (p<0.05)",
      "Sky blue: Borderline", "White: Not significant"),
      fill = c("Cyan", "deepskyblue3", "white"), cex = 0.8, bty = "n")
# 3. Sen's Slope Plot
plot(summer_regcoeff_raster,
      main = "Summer NDVI: Sen's Slope (Change/year, Apr-Sep)",
      col = colorRampPalette(c("darkred", "white", "darkgreen"))(100),
      zlim = slope_zlim,
      legend.args = list(text = "Slope", side = 4, font = 2, line = 2.5, cex = 0.8))
legend("bottomleft", legend = c("Dark red: Strong decline", "White: Stable", "Dark green: Strong increase"),
      fill = c("darkred", "white", "darkgreen"), cex = 0.8, bty = "n")
# Reset layout
par(mfrow = c(1, 1))
# 7. Export the Rasters to GeoTIFF Files
writeRaster(summer_tau_raster, filename = "summer_tau_raster.tif", format = "GTiff", overwrite = TRUE)
writeRaster(summer_pval_raster, filename = "summer_pval_raster.tif", format = "GTiff", overwrite = TRUE)
writeRaster(summer_regcoeff_raster, filename = "summer_regcoeff_raster.tif", format = "GTiff", overwrite = TRUE)
# 8. Comparison with Full-Year Results
full_year_slope <- raster("full_regcoeff_raster.tif")
full_year_pval <- raster("full_pval_raster.tif")
summer_slope <- resample(summer_regcoeff_raster, full_year_slope, method = "bilinear")
summer_pval <- resample(summer_pval_raster, full_year_pval, method = "bilinear")
p_thresh <- if (max(values(full_year_pval), na.rm = TRUE) > 1) 5 else 0.05
full_year_sig <- full_year_pval < p_thresh
summer_sig <- summer_pval < p_thresh
slope_diff <- summer_slope - full_year_slope
sig_change <- overlay(full_year_sig, summer_sig, fun=function(fy, sm){
  out <- rep(NA,length(fy))
  out[!is.na(fy) & !is.na(sm) & fy==sm] <- 0
  out[!is.na(fy) & !is.na(sm) & fy==0 & sm==1] <- 1
  out[!is.na(fy) & !is.na(sm) & fy==1 & sm==0] <- -1
  return(out)
})

```

```

# Mask slopes for significant pixels
summer_slope_sig <- mask(summer_slope, summer_sig, maskvalue = FALSE)
full_year_slope_sig <- mask(full_year_slope, full_year_sig, maskvalue = FALSE)
slope_diff_sig <- mask(slope_diff, full_year_sig | summer_sig, maskvalue = FALSE)
# Load library
library(raster)
# Define custom diverging color palette
col_diff <- colorRampPalette(c("darkred", "white", "darkgreen"))(100)
# Set color scale centered around 0
slope_diff_range <- range(values(slope_diff), na.rm = TRUE)
max_abs <- max(abs(slope_diff_range))
zlim_diff <- c(-max_abs, max_abs)
# Plot with improved settings
plot(slope_diff,
      main = "NDVI Slope Difference (Summer - Full-Year)",
      col = col_diff,
      zlim = zlim_diff,
      legend.args = list(
        text = "Slope Difference (Change in NDVI/year)",
        side = 4, font = 2, line = 2.5, cex = 0.8)
)
# Add custom legend manually
legend("bottomleft",
      legend = c("Strong decline in summer", "No change", "Strong increase in summer"),
      fill = c("darkred", "white", "darkgreen"),
      border = "black",
      bty = "n",
      cex = 0.9)
# 3-category significance change plot
plot(sig_change, col=c("blue","white","red"), main="Change in Significance")
legend("topright", legend=c("Lost","Unchanged","Gained"), fill=c("blue","white","red"), border="black", bg="white")
# 9. Summary Stats
cat("Stronger summer slope:", sum(values(slope_diff) > 0, na.rm = TRUE), "\n")
cat("Weaker summer slope:", sum(values(slope_diff) < 0, na.rm = TRUE), "\n")
cat("Gained significance:", sum(values(sig_change) == 1, na.rm = TRUE), "\n")
cat("Lost significance:", sum(values(sig_change) == -1, na.rm = TRUE), "\n")
hist(values(slope_diff), breaks=50, col="lightgreen", main="Slope Differences", xlab="Summer - Full-Year Slope")
abline(v=0, col="red", lwd=2)
#Q.3. Is there evidence for a slowdown of greening trends in the last years?
1. Load Libraries and data
library(raster)
library(trend)
library(progress)
install.packages("trend")
library(trend)
?seasonalMannKendall # This will bring up the help documentation if the function is loaded correctly
library(trend)
ls("package:trend")
setwd("D:/Pattern_Recognition/Portfolios/Portfolio_1")
# Load the GIMMS NDVI time series raster (monthly data from 1982-2022)
raster_stack <- stack("GIMMS_monthly_1982-2022.tif")
# Try loading the raster stack
raster_stack <- tryCatch({
  stack("GIMMS_monthly_1982-2022.tif")
}, error = function(e) {
  cat("Error loading raster stack:", e$message, "\n")
  NULL
})
# If loaded successfully, check the properties
if (!is.null(raster_stack)) {
  print(raster_stack)
} else {
  cat("Failed to load raster stack.\n")
}
# Check the number of layers to ensure correct loading
n_layers <- nlayers(raster_stack)
print(n_layers) # Should print 492 layers (for 492 months)
#2. Split the Raster into Two Time Periods
# Define the split points for the two periods
end_first_period <- 180 # First period (1983-1998) - 180 months (1983-1998)
start_second_period <- 300 # Second period (2007-2022) - start from January 2007
# Split the raster stack into the two periods
raster_stack_part1 <- raster_stack[[2:end_first_period]] # First period (1983-1998)

```

```
raster_stack_part2 <- raster_stack[[start_second_period:(start_second_period + 19)]] # Second period (2007-2022)
```

#3. Perform the Seasonal Mann-Kendall Test for Both Periods

```
# Process for 1983-1998
```

```
# Convert NDVI raster for the first period to array for analysis
```

```
cube_part1 <- as.array(raster_stack_part1)
```

```
dims_part1 <- dim(cube_part1) # Get dimensions (rows, cols, time layers)
```

```
nrows <- dims_part1[1] # Number of rows
```

```
ncols <- dims_part1[2] # Number of columns
```

```
ntimes <- dims_part1[3] # Number of time layers
```

```
library(trend)
```

4. Initialize empty arrays to store tau and p-values for the first period

```
tau_array_part1 <- array(NA, c(nrows, ncols))
```

```
pval_array_part1 <- array(NA, c(nrows, ncols))
```

5. Loop through each pixel to calculate Seasonal Mann-Kendall statistics

```
pb <- winProgressBar(title = "Progress (1983-1998)", min = 0, max = nrows, width = 300)
```

```
for (i in 1:nrows) {
```

```
  setWinProgressBar(pb, i, title = paste(round(i/nrows*100, 0), "% done"))
```

```
  for (j in 1:ncols) {
```

```
    x <- cube_part1[i, j, ]
```

```
    # Skip if the time series has only NAs
```

```
    if (all(is.na(x))) {
```

```
      tau_array_part1[i, j] <- NA
```

```
      pval_array_part1[i, j] <- NA
```

```
    } else {
```

```
      x_clean <- na.omit(ts(x, start = c(1983, 1), frequency = 12)) # Remove NAs
```

```
      # Check if the cleaned time series has enough data and variance
```

```
      if (length(x_clean) < 3) { # Ensure at least a few data points
```

```
        tau_array_part1[i, j] <- NA
```

```
        pval_array_part1[i, j] <- NA
```

```
      } else if (var(x_clean, na.rm = TRUE) == 0) { # No variance in the time series
```

```
        tau_array_part1[i, j] <- NA
```

```
        pval_array_part1[i, j] <- NA
```

```
      } else {
```

```
        # Perform the Seasonal Mann-Kendall test inside tryCatch
```

```
        res <- tryCatch({
```

```
          smk.test(x_clean)
```

```
        }, error = function(e) {
```

```
          # If an error occurs, return NULL
```

```
          return(NULL)
```

```
        })
```

```
        if (!is.null(res)) {
```

```
          # Extract S and p-value from the result
```

```
          tau_array_part1[i, j] <- res$estimates["S"] # Extract S statistic
```

```
          pval_array_part1[i, j] <- res$p.value # p-value
```

```
        } else {
```

```
          tau_array_part1[i, j] <- NA
```

```
          pval_array_part1[i, j] <- NA
```

```
        }
```

```
      }
```

```
    }
```

```
  }
```

```
}
```

```
close(pb)
```

6. Convert arrays to raster and save them

```
tau_part1_raster <- raster(tau_array_part1, xmn = extent(raster_stack_part1)[1],
```

```
  xmx = extent(raster_stack_part1)[2], ymn = extent(raster_stack_part1)[3],
```

```
  ymx = extent(raster_stack_part1)[4], crs = projection(raster_stack_part1))
```

```
pval_part1_raster <- raster(pval_array_part1, xmn = extent(raster_stack_part1)[1],
```

```
  xmx = extent(raster_stack_part1)[2], ymn = extent(raster_stack_part1)[3],
```

```
  ymx = extent(raster_stack_part1)[4], crs = projection(raster_stack_part1))
```

Save the results for the first period

```
writeRaster(tau_part1_raster, "tau_part1.tif", format = "GTiff", overwrite = TRUE)
```

```
writeRaster(pval_part1_raster, "pval_part1.tif", format = "GTiff", overwrite = TRUE)
```

#7. Plotting

```
# Define a custom color palette: red for negative values (browning), green for positive values
```

```
col_pal <- colorRampPalette(c("red", "white", "green"))(100)
```

```
# Plot tau values for the first period (1983-1998)
```

```
# Add a custom legend and interpret greening
```

```
plot(tau_part1_raster, main = "Tau (1983-1998) - Greening Trends",
```

```
  col = col_pal, # Use custom palette
```

```
  legend = TRUE,
```

```
  axes = TRUE,
```

```
  box = FALSE)
```



```

mtext("Positive tau = Greening (Increased NDVI)", side = 3, line = -1, cex = 0.8)
# Define custom color palette: blue for low p-values (significant trends), red for high p-values (non-significant)
col_pal_pvalue <- colorRampPalette(c("blue", "white", "red"))(100)
# Plot p-value values for the first period (1983-1998)
# Highlight statistical significance (p-value < 0.05)
plot(pval_part1_raster, main = "P-value (1983-1998)",
     col = col_pal_pvalue, # Use the new color palette
     legend = FALSE, # Turn off the default legend
     axes = TRUE,
     box = FALSE)

# Add custom legend
legend("topright",
      legend = c("Significant (p-value < 0.05)", "Non-Significant (p-value ≥ 0.05)"),
      fill = c("blue", "red"),
      title = "P-value Interpretation",
      cex = 0.8,
      box.lty = 1)

# 8. Process for 2007-2022
# Convert NDVI raster for the second period (2007-2022) to array for analysis
cube_part2 <- as.array(raster_stack_part2)
dims_part2 <- dim(cube_part2) # Get dimensions (rows, cols, time layers)
nrows <- dims_part2[1] # Number of rows
ncols <- dims_part2[2] # Number of columns
ntimes <- dims_part2[3] # Number of time layers
# Initialize empty arrays to store tau and p-values for the second period
tau_array_part2 <- array(NA, c(nrows, ncols))
pval_array_part2 <- array(NA, c(nrows, ncols))
# Loop through each pixel to calculate Seasonal Mann-Kendall statistics for the second period
pb <- winProgressBar(title = "Progress (2007-2022)", min = 0, max = nrows, width = 300)
for (i in 1:nrows) {
  setWinProgressBar(pb, i, title = paste(round(i / nrows * 100, 0), "% done"))
  for (j in 1:ncols) {
    # Extract the time series for the current pixel (i, j)
    x <- cube_part2[i, j, ]
    # Skip if the time series has only NAs
    if (all(is.na(x))) {
      tau_array_part2[i, j] <- NA
      pval_array_part2[i, j] <- NA
    } else {
      # Clean the time series by removing NAs
      x_clean <- na.omit(ts(x, start = c(2007, 1), frequency = 12)) # Remove NAs

      # Check if the cleaned time series has enough data and variance
      if (length(x_clean) < 3) { # Ensure at least a few data points
        tau_array_part2[i, j] <- NA
        pval_array_part2[i, j] <- NA
      } else if (var(x_clean, na.rm = TRUE) == 0) { # No variance in the time series
        tau_array_part2[i, j] <- NA
        pval_array_part2[i, j] <- NA
      } else {
        # Perform the Seasonal Mann-Kendall test inside tryCatch
        res <- tryCatch({
          smk.test(x_clean) # Run the Seasonal Mann-Kendall test
        }, error = function(e) {
          # If an error occurs, return NULL
          return(NULL)
        })
        if (!is.null(res)) {
          # Extract S (Tau equivalent) and p-value from the result
          tau_array_part2[i, j] <- res$estimates["S"] # Extract S statistic (tau)
          pval_array_part2[i, j] <- res$p.value # p-value
        } else {
          tau_array_part2[i, j] <- NA
          pval_array_part2[i, j] <- NA
        }
      }
    }
  }
}
close(pb)
# Convert arrays to raster and save them for the second period
tau_part2_raster <- raster(tau_array_part2, xmn = extent(raster_stack_part2)[1],

```

```

      xmx = extent(raster_stack_part2)[2], ymn = extent(raster_stack_part2)[3],
      ymx = extent(raster_stack_part2)[4], crs = projection(raster_stack_part2))
pval_part2_raster <- raster(pval_array_part2, xmn = extent(raster_stack_part2)[1],
      xmx = extent(raster_stack_part2)[2], ymn = extent(raster_stack_part2)[3],
      ymx = extent(raster_stack_part2)[4], crs = projection(raster_stack_part2))
# Save the results for the second period
writeRaster(tau_part2_raster, "tau_part2.tif", format = "GTiff", overwrite = TRUE)
writeRaster(pval_part2_raster, "pval_part2.tif", format = "GTiff", overwrite = TRUE)
# Convert arrays to raster and save them for the second period
tau_part2_raster <- raster(tau_array_part2, xmn = extent(raster_stack_part2)[1],
      xmx = extent(raster_stack_part2)[2], ymn = extent(raster_stack_part2)[3],
      ymx = extent(raster_stack_part2)[4], crs = projection(raster_stack_part2))
pval_part2_raster <- raster(pval_array_part2, xmn = extent(raster_stack_part2)[1],
      xmx = extent(raster_stack_part2)[2], ymn = extent(raster_stack_part2)[3],
      ymx = extent(raster_stack_part2)[4], crs = projection(raster_stack_part2))
# Save the results for the second period
writeRaster(tau_part2_raster, "tau_part2.tif", format = "GTiff", overwrite = TRUE)
writeRaster(pval_part2_raster, "pval_part2.tif", format = "GTiff", overwrite = TRUE)
# Plotting
# Define a custom color palette for tau values: red = browning, green = greening
col_pal_tau <- colorRampPalette(c("red", "white", "green"))(100)
plot(tau_part2_raster,
      main = "Tau (2007–2022)",
      col = col_pal_tau,
      legend = TRUE,
      axes = TRUE,
      box = FALSE)
legend("topright",
      legend = c("Greening (Positive Tau)", "Browning (Negative Tau)"),
      fill = c("green", "red"),
      title = "Trend Interpretation",
      cex = 0.8,
      box.lty = 1)
# Define a custom color palette for p-values: blue for low p-values (significant trends), red for high p-values (non-significant)
col_pal_pvalue <- colorRampPalette(c("blue", "white", "red"))(100)
# Plot p-value values for the second period (2007-2022) - Statistical Significance
plot(pval_part2_raster,
      main = "P-value (2007-2022) - Statistical Significance",
      col = col_pal_pvalue, # Use the custom color palette for p-values
      legend = FALSE, # Turn off the default legend
      axes = TRUE,
      box = FALSE)
legend("topright",
      legend = c("Significant (p-value < 0.05)", "Non-Significant (p-value ≥ 0.05)"),
      fill = c("blue", "red"),
      title = "P-value Interpretation",
      cex = 0.8,
      box.lty = 1)
# Now, compare tau values for both periods to check for a slowdown in greening
tau_difference <- tau_array_part2 - tau_array_part1
# Create a raster to visualize the difference in tau values
tau_diff_raster <- raster(tau_difference, xmn = extent(raster_stack_part2)[1],
      xmx = extent(raster_stack_part2)[2], ymn = extent(raster_stack_part2)[3],
      ymx = extent(raster_stack_part2)[4], crs = projection(raster_stack_part2))
# Plot the difference in tau values
plot(tau_diff_raster, main = "Difference in Tau (2007-2022 vs 1983-1998)",
      col = rev(heat.colors(100)),
      legend = TRUE, axes = TRUE, box = FALSE)
title(main = "Difference in Greening Trends", line = 0.5)
# Calculate the difference in tau values for both periods (2007-2022 vs 1983-1998)
tau_difference <- tau_array_part2 - tau_array_part1
# Create a raster to visualize the difference in tau values
tau_diff_raster <- raster(tau_difference,
      xmn = extent(raster_stack_part2)[1],
      xmx = extent(raster_stack_part2)[2],
      ymn = extent(raster_stack_part2)[3],
      ymx = extent(raster_stack_part2)[4],
      crs = projection(raster_stack_part2))
# Define a custom diverging color palette to visualize tau difference
# Red = slowdown/browning, Yellow = neutral, Green = increase/greening
col_pal_diff_tau <- colorRampPalette(c("red", "yellow", "green"))(100)
# Plot the difference in tau values (2007-2022 vs 1983-1998) with custom colors
plot(tau_diff_raster,

```

```

    main = "Difference in Tau (2007-2022 vs 1983-1998)",
    col = col_pal_diff_tau, # Use custom diverging color palette
    legend = TRUE,
    axes = TRUE,
    box = FALSE)
# Custom legend for tau difference interpretation
legend("topright",
      legend = c("Slowdown (Less Greening)", "No Significant Change", "Increase in Greening (More Positive Tau)",
        fill = c("red", "yellow", "green"),
        title = "Tau Difference Interpretation",
        cex = 0.8,
        box.lty = 1)
# Calculate the area covered by each category (in km^2 or any other unit)
# Define the resolution of each pixel (in this case, assuming 1km^2 per pixel, adjust as needed)
pixel_area_km2 <- res(tau_diff_raster)[1] * res(tau_diff_raster)[2] # Pixel area in square kilometers
# Convert raster to a data frame for easier calculation
tau_categories_df <- as.data.frame(tau_categories, xy = TRUE, na.rm = TRUE)
colnames(tau_categories_df) <- c("x", "y", "category")
# Calculate the area for each category
area_slowdown <- sum(tau_categories_df$category == 1) * pixel_area_km2
area_no_change <- sum(tau_categories_df$category == 2) * pixel_area_km2
area_increase <- sum(tau_categories_df$category == 3) * pixel_area_km2
#Summary
cat("Summary of Greening Trends:\n")
cat("Area showing Slowdown (Less Greening):", area_slowdown, "km^2\n")
cat("Area showing No Significant Change:", area_no_change, "km^2\n")
cat("Area showing Increase in Greening:", area_increase, "km^2\n")
# Interpretation based on printed output:
if (area_slowdown > area_increase) {
  cat("There is evidence for a slowdown of greening trends in the last years.")
} else {
  cat("There is no significant slowdown in greening trends in the last years.")
}
# Q.4. Which land-use/cover class (CORINE) contributes most to the effect of a greening trend?
# Load required libraries
library(terra); library(dplyr); library(ggplot2); library(RColorBrewer); library(readxl)
setwd("D:/Pattern_Recognition/Portfolios/Portfolio_1")
# 1. Load CORINE legend data
clc_legend <- read_excel("clc_legend.xls") %>%
  select(CLC_CODE, LABEL1) # Level 1 land cover descriptions
# 2. Load and preprocess raster data
corine_raster <- rast("clc8km_clip1.tif")
tau_raster <- rast("outputs/full_tau_raster.tif") # NDVI greening trend raster
# Align CRS of NDVI raster to CORINE raster
tau_proj <- project(tau_raster, crs(corine_raster))
# Resample CORINE raster to match NDVI raster resolution and extent
corine_resampled <- resample(corine_raster, tau_proj, method = "near")
# 3. Extract raster values and filter
g_vals <- values(tau_proj)
c_vals <- values(corine_resampled)
# Keep only valid (non-NA) pixels
valid <- !is.na(g_vals) & !is.na(c_vals)
g_vals <- g_vals[valid]
c_vals <- c_vals[valid]
# Create data frame and filter positive greening values only
trend_data <- data.frame(
  greening = g_vals,
  clc_code = c_vals
) %>%
  filter(greening > 0)
# 4. Manual mapping from Level 3 codes to Level 1 labels
mapping <- data.frame(
  clc_code = c(111:142, 211:244, 311:335, 411:423, 511:523),
  level1_label = c(
    rep("Artificial surfaces", length(111:142)),
    rep("Agricultural areas", length(211:244)),
    rep("Forest & semi-natural", length(311:335)),
    rep("Wetlands", length(411:423)),
    rep("Water bodies", length(511:523))
  )
)
# Join mapping to trend data
trend_with_level1 <- trend_data %>%

```

```

left_join(mapping, by = "clc_code")
# Check for missing mappings
missing_mapped <- trend_with_level1 %>% filter(is.na(level1_label))
cat("Number of greening pixels with missing Level-1 mapping:", nrow(missing_mapped), "\n")
# 5. Summarize greening pixels count by Level 1 class
greening_by_level1 <- trend_with_level1 %>%
  filter(!is.na(level1_label)) %>%
  group_by(level1_label) %>%
  summarize(count = n(), .groups = "drop")
# 6. Define desired order for factor levels
desired_order <- c(
  "Artificial surfaces",
  "Agricultural areas",
  "Forest & semi-natural",
  "Wetlands",
  "Water bodies"
)
greening_by_level1$level1_label <- factor(
  greening_by_level1$level1_label,
  levels = desired_order
)
# 7. Plot bar chart
ggplot(greening_by_level1, aes(x = level1_label, y = count, fill = level1_label)) +
  geom_col(color = "black", size = 0.7, alpha = 0.85) + # Bars with black outline
  scale_fill_brewer(palette = "Set2") + # Soft pastel color palette
  geom_text(aes(label = format(count, big.mark = ",")), # Show counts on top of bars
    vjust = -0.8, size = 4, fontface = "bold", color = "#444444") +
  labs(
    x = "CORINE Level 1 Land-Cover Class",
    y = "Count of Greening Pixels",
    title = "Greening Trend by Broad Land-Cover Class"
  ) +
  theme_minimal(base_family = "Segoe UI") +
  theme(
    axis.text.x = element_text(angle = 30, hjust = 1, size = 12, face = "bold"),
    axis.title = element_text(size = 14, face = "bold"),
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold", color = "#2E4057"),
    legend.position = "none",
    panel.grid.major.y = element_line(color = "gray90"),
    panel.grid.major.x = element_blank()
  ) +
  expand_limits(y = max(greening_by_level1$count) * 1.1) # Add space on top for text

```

R code for Portfolio 2



Final

Syndrome_Mapping_c

```

Required libraries
library(terra)
library(dplyr)
library(ggplot2)
library(RColorBrewer)
library(pbapply)

#-----
# Settings
#-----
setwd("E:/pattern recognition/port2_folio")

#2. Set parallel processing
# -----
terraOptions()
parallel::detectCores()
#terra::setCores(4) # Set number of cores for parallel processing
terraOptions(progress = TRUE) # Show progress bars in terra functions

```

```

#-----
# 1. Load NDVI stack (GIMMS-like)
#-----
ndvi_file <- "E:/pattern recognition/Potfolio_part2/GIMMS_monthly_1982-2022.tif"
ndvi_stack <- rast(ndvi_file)
months_per_year <- 12

# Scale NDVI immediately after loading
ndvi_stack <- ndvi_stack / 10000

# Check value range
summary(values(ndvi_stack)) # should now be between -1 and 1

print(crs(ndvi_stack))
print(res(ndvi_stack))

#-----
# 2. Load CORINE and reproject/reclassify -> aggregate -> align
#-----
corine_file <- "E:/pattern recognition/Portfolio_Part_I/clc8km_clip1.tif"
corine <- rast(corine_file)
print(crs(corine))
# Reproject CORINE to NDVI CRS (nearest neighbour because categorical)
corine_reproj <- project(corine, ndvi_stack, method="near")

# Reclassify into Agriculture (1) and Natural (2)
# Adjust the ranges to your CORINE codebook if needed
rcl <- matrix(c(
  210, 219, 1,
  220, 229, 1,
  300, 399, 2,
  400, 499, 2,
  500, 599, 2
), ncol=3, byrow=TRUE)
corine_reclass <- classify(corine_reproj, rcl=rcl)

# corine_reclass is your reclassified raster
unique_vals <- unique(values(corine_reclass))
unique_vals

table(values(corine_reclass), useNA="ifany")

#-----
# Mask unwanted codes, keep only Agriculture/Natural/NA
# Keep 1 → 1, 2 → 2, everything else → NA
corine_clean <- ifel(corine_reproj == 1, 1,
  ifel(corine_reproj == 2, 2, NA))

# Set RAT / levels (helpful when converting to factors later)
#levels(corine_reclass) <- data.frame(ID = c(1,2), land_cover = c("Agriculture","Natural"))
levels(corine_clean) <- data.frame(
  ID = c(1,2),
  land_cover = c("Agriculture","Natural")
)
#-----
# Aggregate CORINE to NDVI resolution if finer
fact_xy <- round(res(corine_clean) / res(ndvi_stack))
if (any(fact_xy > 1)) {
  corine_agg <- aggregate(corine_clean, fact = fact_xy, fun = modal)
} else {
  corine_agg <- corine_clean
}

# Align exactly to NDVI grid (resample nearest)
corine_aligned <- resample(corine_agg, ndvi_stack, method = "near")
corine_clean <- corine_aligned # use this from now on

compareGeom(ndvi_stack, corine_clean, stopOnError = FALSE)
cat("CORINE NAs:", sum(is.na(values(corine_clean))), "\n")

#-----

```

```

# 3. Fourier-based phenology (per year): PeakMonth, MaxNDVI, MeanNDVI
#   fourier_metrics returns numeric vector length 3
#-----
fourier_metrics <- function(ts){
  if (all(is.na(ts))) return(c(NA, NA, NA))
  n_years <- length(ts) %/% months_per_year
  if (n_years < 1) return(c(NA, NA, NA))
  ts_matrix <- matrix(ts[1:(n_years*months_per_year)],
    nrow = n_years, ncol = months_per_year, byrow = TRUE)
  peak_months <- numeric(n_years)
  max_ndvi <- numeric(n_years)
  mean_ndvi <- numeric(n_years)
  for (i in 1:n_years){
    y <- ts_matrix[i, ]
    t <- 1:12
    if (all(is.na(y))) { peak_months[i] <- NA; max_ndvi[i] <- NA; mean_ndvi[i] <- NA; next }
    fit <- tryCatch(lm(y ~ cos(2*pi*t/12) + sin(2*pi*t/12)), error = function(e) NULL)
    if (is.null(fit)) { peak_months[i] <- NA; max_ndvi[i] <- NA; mean_ndvi[i] <- NA; next }
    fitted_vals <- predict(fit)
    peak_months[i] <- which.max(fitted_vals)
    max_ndvi[i] <- max(fitted_vals, na.rm = TRUE)
    mean_ndvi[i] <- mean(fitted_vals, na.rm = TRUE)
  }
  c(mean(peak_months, na.rm = TRUE), mean(max_ndvi, na.rm = TRUE), mean(mean_ndvi, na.rm = TRUE))
}

# Apply across the NDVI stack
phenology_raster <- app(ndvi_stack, fun = fourier_metrics)
# phenology_raster now has 3 layers (layer1=PeakMonth, layer2=MaxNDVI, layer3=MeanNDVI)

names(phenology_raster) <- c("PeakMonth", "MaxNDVI", "MeanNDVI")

#check ndvi values
summary(values(ndvi_stack))

#-----
# 4. Trend of yearly mean NDVI (slope + p-value significance)
#   trend_fun returns -1 (sig dec), 0 (ns), 1 (sig inc)
#-----
trend_fun <- function(x){
  if (all(is.na(x))) return(NA)
  n_years <- length(x) %/% months_per_year
  ts_matrix <- matrix(x[1:(n_years*months_per_year)],
    nrow = n_years, ncol = months_per_year, byrow = TRUE)
  yearly_mean <- rowMeans(ts_matrix, na.rm = TRUE)
  if (sum(!is.na(yearly_mean)) < 5) return(NA) # need at least 5 years
  year_seq <- seq_along(yearly_mean)
  fit <- tryCatch(lm(yearly_mean ~ year_seq), error = function(e) NULL)
  if (is.null(fit)) return(NA)
  slope <- coef(fit)[2]
  p_val <- summary(fit)$coefficients[2, 4]
  if (is.na(p_val)) return(0)
  if (p_val < 0.05) {
    if (slope > 0) return(1)
    if (slope < 0) return(-1)
  }
  return(0)
}

slope_significant_class <- app(ndvi_stack, fun = trend_fun)
names(slope_significant_class) <- "NDVI_trend_class" # values -1,0,1

#-----
# 5. Peak month per year series and trend of peak month
#-----
peak_month_fun <- function(ts){
  if (all(is.na(ts))) return(rep(NA, length(ts) / 12))
  n_years <- length(ts) %/% months_per_year
  ts_matrix <- matrix(ts[1:(n_years*months_per_year)], nrow=n_years, ncol=months_per_year, byrow=TRUE)
  peak_months <- numeric(n_years)
  for (i in 1:n_years){
    y <- ts_matrix[i, ]; t <- 1:12
    if (all(is.na(y))) { peak_months[i] <- NA; next }

```

```

    fit <- tryCatch(lm(y ~ cos(2*pi*t/12) + sin(2*pi*t/12)), error=function(e) NULL)
    if (is.null(fit)) { peak_months[i] <- NA; next }
    peak_months[i] <- which.max(predict(fit))
  }
  peak_months
}

peak_month_stack <- app(ndvi_stack, fun = peak_month_fun)
# trend of peak month (linear trend of month number)
peak_trend_fun <- function(pm_ts){
  if (all(is.na(pm_ts))) return(NA)
  if (sum(!is.na(pm_ts)) < 5) return(NA)
  fit <- tryCatch(lm(pm_ts ~ seq_along(pm_ts)), error=function(e) NULL)
  if (is.null(fit)) return(NA)
  coef(fit)[2]
}
peak_trend_raster <- app(peak_month_stack, fun = peak_trend_fun)
names(peak_trend_raster) <- "PeakMonth_trend"

# Classify peak month trend into -1/0/1 using thresholds (adjust thresholds if needed)
peak_trend_class <- classify(peak_trend_raster,
  rcl = matrix(c(-Inf, -0.1, -1,
                -0.1, 0.1, 0,
                0.1, Inf, 1), ncol=3, byrow=TRUE))
names(peak_trend_class) <- "Peak_trend_class"

#-----
# 6. Ensure CRS consistency and resample trend rasters to CORINE grid
#-----
# Force CRSs if necessary
crs(slope_significant_class) <- crs(ndvi_stack)
crs(peak_trend_class) <- crs(ndvi_stack)

# Resample to corine_reclass grid (nearest for categorical)
trend_resampled <- resample(slope_significant_class, corine_clean, method = "near")
peak_trend_resampled <- resample(peak_trend_class, corine_clean, method = "near")

# Crop / mask to CORINE extent to ensure exact overlap
trend_resampled <- crop(trend_resampled, corine_clean)
peak_trend_resampled <- crop(peak_trend_resampled, corine_clean)

compareGeom(corine_clean, trend_resampled, peak_trend_resampled, stopOnError = FALSE)

#-----
# 7. Build combined syndrome raster
# Schematic: ID = lc*100 + (ndvi_tr + 1)*10 + (peak_tr + 1)
# where lc in {1=Agriculture,2=Natural}; ndvi_tr in {-1,0,1}; peak_tr in {-1,0,1}
#-----
# stack and apply
stack_for_syndrome <- c(corine_clean, trend_resampled, peak_trend_resampled)
# safety: convert to numeric to avoid factor weirdness
stack_for_syndrome <- app(stack_for_syndrome, fun=function(x) as.numeric(x))

# Define valid mask based on all input rasters
valid_mask <- !is.na(corine_clean) & !is.na(trend_resampled) & !is.na(peak_trend_resampled)

lc_valid <- mask(corine_clean, valid_mask, maskvalue=FALSE)
trend_valid <- mask(trend_class, valid_mask, maskvalue=FALSE)
peak_valid <- mask(peak_trend_resampled, valid_mask, maskvalue=FALSE)

combined_syndrome <- app(stack_for_syndrome, fun = function(x){
  lc <- x[1]; ndvi_tr <- x[2]; peak_tr <- x[3]
  if (any(is.na(c(lc, ndvi_tr, peak_tr)))) return(NA)
  ndvi_tr_shift <- ndvi_tr + 1
  peak_tr_shift <- peak_tr + 1
  id <- lc * 100 + ndvi_tr_shift * 10 + peak_tr_shift
  return(id)
})

combined_syndrome <- as.factor(combined_syndrome)
#=====
combined_syndrome <- app(c(corine_reclass, trend_resampled, peak_trend_resampled), fun = function(x) {

```

```

if (any(is.na(x))) return(NA)
lc <- x[1]; ndvi_tr <- x[2]; peak_tr <- x[3]
ndvi_tr_shift <- ndvi_tr + 1
peak_tr_shift <- peak_tr + 1
lc*100 + ndvi_tr_shift*10 + peak_tr_shift
})

summary(values(combined_syndrome))
length(unique(values(combined_syndrome)))

#=====

# Build RAT (lookup table) - IDs for all possible combos
syndrome_labels <- c(
  "Agriculture - NDVI Decreasing - Peak Earlier", "Agriculture - NDVI Decreasing - Peak Stable",
  "Agriculture - NDVI Decreasing - Peak Later", "Agriculture - NDVI Stable - Peak Earlier",
  "Agriculture - NDVI Stable - Peak Stable", "Agriculture - NDVI Stable - Peak Later",
  "Agriculture - NDVI Increasing - Peak Earlier", "Agriculture - NDVI Increasing - Peak Stable",
  "Agriculture - NDVI Increasing - Peak Later", "Natural - NDVI Decreasing - Peak Earlier",
  "Natural - NDVI Decreasing - Peak Stable", "Natural - NDVI Decreasing - Peak Later",
  "Natural - NDVI Stable - Peak Earlier", "Natural - NDVI Stable - Peak Stable",
  "Natural - NDVI Stable - Peak Later", "Natural - NDVI Increasing - Peak Earlier",
  "Natural - NDVI Increasing - Peak Stable", "Natural - NDVI Increasing - Peak Later"
)
ids_all <- c(111,112,113,121,122,123,131,132,133,
            211,212,213,221,222,223,231,232,233)
rat_all <- data.frame(ID = as.character(ids_all), Label = syndrome_labels, stringsAsFactors = FALSE)

# Keep only present IDs in rat for neatness
ids_present <- sort(unique(values(combined_syndrome)))
ids_present <- ids_present[!is.na(ids_present)]
#rat_combined <- rat_all[rat_all$ID %in% as.character(ids_present), , drop = FALSE]

rat_combined <- rat_all[rat_all$ID %in% ids_present, , drop = FALSE]
rat_combined$ID <- as.numeric(rat_combined$ID)
levels(combined_syndrome) <- rat_combined[, c("ID", "Label")]

levels(combined_syndrome)

levels(combined_syndrome)
freq(combined_syndrome)
#=====

#1) Check how many IDs are in your raster
ids_present <- sort(unique(values(combined_syndrome)))
ids_present <- ids_present[!is.na(ids_present)]
length(ids_present)
ids_present
#Step 1 — Confirm raster validity
combined_syndrome
summary(values(combined_syndrome))

compareGeom(corine_reclass, trend_resampled, peak_trend_resampled, stopOnError = FALSE)

#-----
# 8. Convert to dataframe for plotting and analysis
#-----
combined_df <- as.data.frame(combined_syndrome, xy = TRUE, na.rm = FALSE)
colnames(combined_df)[3] <- "Label"
#check plot properties
nrow(combined_df)
table(combined_df$Label, useNA = "ifany")

unique(values(combined_syndrome))

# palette
n_labels <- length(unique(combined_df$Label))
combined_colors <- brewer.pal(min(12, n_labels), "Paired")
combined_colors <- rep(combined_colors, length.out = n_labels)

ggplot(combined_df, aes(x = x, y = y, fill = Label)) +

```



```

geom_raster() +
scale_fill_manual(values = combined_colors, drop = FALSE) +
coord_fixed() +
theme_minimal() +
theme(axis.text = element_blank(), axis.ticks = element_blank(),
      axis.title = element_blank(), plot.title = element_text(face = "bold", size = 12)) +
labs(title = "Combined Phenology Syndrome Map", fill = "Syndrome")

#-----
# 9. Export raster + RAT
#-----
output_dir <- "E:/pattern recognition/port2_folio/phenology_metrics/"
dir.create(output_dir, recursive = TRUE, showWarnings = FALSE)

writeRaster(combined_syndrome,
            filename = file.path(output_dir, "combined_phenology_syndrome.tif"),
            overwrite = TRUE)

write.csv(rat_combined,
          file = file.path(output_dir, "combined_phenology_syndrome_labels.csv"),
          row.names = FALSE)

cat("Workflow complete! Raster and RAT exported.\n")

#-----
# 10. Compute dominance (frequency & area)
#-----
# Frequency table
library(dplyr)

# Re-run frequency extraction
freq_tbl <- freq(combined_syndrome)
head(freq_tbl)

# Step 1 — rename columns for clarity
colnames(freq_tbl) <- c("layer", "Label", "count")

# Drop the 'layer' column
freq_df <- freq_tbl[, c("Label", "count")]
# Compute area
cell_area_km2 <- prod(res(combined_syndrome)) / 1e6
freq_df$area_km2 <- freq_df$count * cell_area_km2

head(freq_df)

#visualize dominance
library(ggplot2)

ggplot(freq_df, aes(x = reorder(Label, -area_km2), y = area_km2, fill = Label)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  labs(
    title = "Dominant Phenology Syndromes",
    x = "Syndrome",
    y = "Area (km²)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
output_dir <- "E:/pattern recognition/part2_redone/phenology_metrics/"
dir.create(output_dir, recursive = TRUE, showWarnings = FALSE)
write.csv(freq_df,
          file = paste0(output_dir, "phenology_syndrome_summary.csv"),
          row.names = FALSE)
cat("✅ Syndrome frequency summary exported to:\n",
    paste0(output_dir, "phenology_syndrome_summary.csv"), "\n")

```

R code for Portfolio 3

R code for Portfolio 3

Portfolio Part III - R Code

```

# Distributed Lag Model (DLM) + Random Forest Analysis
# Based on MODIS NDVI and Rainfall time series in Botswana
# 1. Load libraries
library(terra)
library(forecast)
library(tseries)
library(lmtest)
library(dplyr)
library(randomForest)
library(ranger)
# 2. Load NDVI and Rainfall raster stacks
ndvi_stack <- rast("Botswana_NDVI_USGS.tif")
rain_stack <- rast("rs_rain_chirps_Botswana.tif")
# 3. Pick a sample pixel for time series analysis
sample_point <- cbind(25.5, -22.0)
ndvi_ts <- extract(ndvi_stack, sample_point)[1, -1]
rain_ts <- extract(rain_stack, sample_point)[1, -1]
# 4. Create time series objects
ndvi <- ts(as.numeric(ndvi_ts), start=c(2000,1), frequency=12)
rain <- ts(as.numeric(rain_ts), start=c(2000,1), frequency=12)
# 5. STL decomposition
ndvi_stl <- stl(ndvi, s.window="periodic")
rain_stl <- stl(rain, s.window="periodic")
# 6. Extract anomalies (remainder)
ndvi_anom <- ndvi_stl$time.series[,3]
rain_anom <- rain_stl$time.series[,3]
# 7. Lagged regression (distributed lag model)
lag1 <- stats::lag(rain_anom, -1)
lag2 <- stats::lag(rain_anom, -2)
lag3 <- stats::lag(rain_anom, -3)
lagged_df <- na.omit(data.frame(ndvi_anom, rain_anom, lag1, lag2, lag3))
model_lag <- lm(ndvi_anom ~ rain_anom + lag1 + lag2 + lag3, data=lagged_df)
summary(model_lag)
# 8. Durbin-Watson test for autocorrelation
dwtest(model_lag)
# 9. STL Plot (optional)
plot(ndvi_stl, main="NDVI Decomposition")
plot(rain_stl, main="Rainfall Decomposition")
# Regression Analysis using original time-series (no lag)
cor(as.numeric(ndvi), as.numeric(rain), use = "complete.obs")
# 6. Extract remainder components (anomalies)
ndvi_anom <- ndvi_stl$time.series[, "remainder"]
rain_anom <- rain_stl$time.series[, "remainder"]
# 7. Correlation between rainfall and NDVI anomalies
cor(as.numeric(ndvi_anom), as.numeric(rain_anom), use = "complete.obs")
# 8. Create lag variables (1 and 2 month lags)
lag1 <- stats::lag(rain_anom, -1)
lag2 <- stats::lag(rain_anom, -2)
# 9. Create data frame for regression
lagged_df <- na.omit(data.frame(ndvi_anom, rain_anom, lag1, lag2))
# 10. Fit distributed lag model
model_lag <- lm(ndvi_anom ~ rain_anom + lag1 + lag2, data = lagged_df)
summary(model_lag)
# 11. Durbin-Watson test for autocorrelation in residuals
dwtest(model_lag)
# 12. Plot Residuals and NDVI Anomalies (side-by-side like Figure 7)
ndvi_anom_ts <- ts(ndvi_anom, start = c(2000, 1), frequency = 12)
par(mfrow = c(1, 2))
plot(model_lag$residuals, type = "l", main = "Model Residuals",
      xlab = "Index", ylab = "Residuals")
plot(ndvi_anom_ts, type = "l", main = "NDVI Anomalies",
      xlab = "Time", ylab = "NDVI", ylim = c(0.1, 0.55)) # Adjusted Y-axis
# check correlation
# Model 1: No lag
model0 <- lm(as.numeric(ndvi) ~ as.numeric(rain))
summary(model0)
par(mfrow = c(1, 2))
plot(model0$residuals, type = "l", main = "Residuals (No Lag)", ylab = "residuals", xlab = "Index")
plot(ndvi, main = "NDVI Time Series", ylab = "ndvi", xlab = "Time")

```

```

# Model 2: With 1-month lag (backshift)
model1 <- lm(as.numeric(ndvi) ~ stats::lag(as.numeric(rain), -1))
summary(model1)
par(mfrow = c(1, 2))
plot(model1$residuals, type = "l", main = "Residuals (1-Month Lag)", ylab = "residuals", xlab = "Index")
plot(ndvi, main = "NDVI Time Series", ylab = "ndvi", xlab = "Time")
# Model 3: With 2-month lags (backshift: 0, -1, -2)
model2 <- lm(as.numeric(ndvi) ~
  stats::lag(as.numeric(rain), 0) +
  stats::lag(as.numeric(rain), -1) +
  stats::lag(as.numeric(rain), -2))
summary(model2)
par(mfrow = c(1, 2))
plot(model2$residuals, type = "l", main = "Residuals (2-Month Lag)", ylab = "residuals", xlab = "Index")
plot(ndvi, main = "NDVI Time Series", ylab = "ndvi", xlab = "Time")
# Optional: Durbin-Watson test for serial autocorrelation
library(lmtest)
dwtest(model2)
model <- lm(as.numeric(ndvi) ~ Lag(as.numeric(rain), 1) + Lag(as.numeric(rain), 2)) # considering two lags
summary(model) # even better!
plot(model$residuals, type="l")
plot(ndvi)
par(mfrow=c(1,1))
dwtest(model)
#Model outputs using anomalies.
# Use base R's diagnostic plot for linear models
par(mfrow = c(2, 2)) # Arrange in 2x2 grid
plot(model_lag)
dwtest(model_lag)
summary(model_lag)
# Plot all 4 default diagnostics:
# - Residuals vs Fitted
# - Normal Q-Q
# - Scale-Location
# - Residuals vs Leverage
####model
# Step 1: OLS Regression of NDVI and rainfall anomalies
model <- lm(ndvi_anom ~ rain_anom)
# Step 2: Fit ARIMA model to remove serial correlation in residuals
library(forecast)
optfit <- auto.arima(model$residuals, max.p = 5, max.q = 0) # Only AR terms
# Step 3: Filter original anomaly series using ARIMA residuals
yfiltered <- residuals(Arima(ndvi_anom, model = optfit))
xfiltered <- residuals(Arima(rain_anom, model = optfit))
y_vec <- as.numeric(yfiltered)
x_vec <- as.numeric(xfiltered)
# --- Step 2: Manually create lagged vectors (padded with NA at the start) ---
create_lag <- function(x, k) {
  c(rep(NA, k), x[1:(length(x) - k)])
}
lagx0 <- x_vec
lagx1 <- create_lag(x_vec, 1)
lagx2 <- create_lag(x_vec, 2)
lagx3 <- create_lag(x_vec, 3)
lagx4 <- create_lag(x_vec, 4)
lagx5 <- create_lag(x_vec, 5)
lagx6 <- create_lag(x_vec, 6)
# --- Step 3: Build data frame with aligned series ---
df_lagged <- data.frame(
  ndvi = y_vec,
  Lag0 = lagx0,
  Lag1 = lagx1,
  Lag2 = lagx2,
  Lag3 = lagx3,
  Lag4 = lagx4,
  Lag5 = lagx5,
  Lag6 = lagx6
)
# --- Step 4: Remove rows with NA due to lagging ---

```

```

df_lagged <- na.omit(df_lagged)
# --- Step 5: Fit regression model ---
model_lag6 <- lm(ndvi ~ ., data = df_lagged)
summary(model_lag6)
# --- Step 6: Durbin-Watson test ---
library(lmtest)
dwtest(model_lag6)
# --- Step 7: Diagnostic plots (Figure 8 style) ---Figure 9: Model Graphs for Distributed lag models.
par(mfrow = c(2, 2))
plot(model_lag6)
par(mfrow = c(1, 1)) # Reset
# Step 1: Convert raster stacks to matrices
ndvi_matrix <- as.matrix(values(ndvi_stack)) # rows = pixels, cols = time
rain_matrix <- as.matrix(values(rain_stack)) # same dimensions
F_values <- sapply(1:nrow(ndvi_matrix), function(i) {
  ndvi_pixel <- ndvi_matrix[i, ]
  rainfall_pixel <- rain_matrix[i, ]
  # Skip missing data
  if (all(is.na(ndvi_pixel)) || all(is.na(rainfall_pixel))) return(NA)
  # Pre-whiten the series (optional, or use raw anomalies)
  tryCatch({
    model <- lm(ndvi_pixel ~ rainfall_pixel +
      stats::lag(rainfall_pixel, -1) +
      stats::lag(rainfall_pixel, -2) +
      stats::lag(rainfall_pixel, -3))
    summary(model)$fstatistic[1] # return F-value
  }, error = function(e) NA)
})

# Step 3: Create empty raster and assign F-values
F_raster <- ndvi_stack[[1]] # use first NDVI layer as template
values(F_raster) <- F_values
# Step 4: Save as raster file
writeRaster(F_raster, filename = "F.tif", overwrite = TRUE)
# ----- PART II: Spatial regression using environmental layers -----
library(raster)
library(lattice)
library(gstat)
library(ranger)
library(ggplot2)
library(tibble)
# Load the data:
data <- raster("Botswana_Elevation_resample.tif")
x <- terrain(data, opt = c('slope', 'aspect', 'roughness', 'flowdir', 'TPI', 'TRI'), unit = 'degrees')
plot(x)
# Create grid from elevation raster
grids <- as(data, "SpatialGridDataFrame")
names(grids)[1] <- "elev"
# Ensure cell size consistency
slot(slot(grids, "grid"), "cellsize")[1] <- slot(slot(grids, "grid"), "cellsize")[2]
# Add terrain layers to the grid one by one
temp <- as(x[["slope"]], "SpatialGridDataFrame")
grids[["slope"]] <- temp
temp <- as(x[["aspect"]], "SpatialGridDataFrame")
grids[["aspect"]] <- temp
temp <- as(x[["tri"]], "SpatialGridDataFrame")
grids[["tri"]] <- temp
temp <- as(x[["tpi"]], "SpatialGridDataFrame")
grids[["tpi"]] <- temp
temp <- as(x[["roughness"]], "SpatialGridDataFrame")
grids[["roughness"]] <- temp
temp <- as(x[["flowdir"]], "SpatialGridDataFrame")
grids[["flowdir"]] <- temp
# Import additional layers into Spatial Grid
# Altitude above channel network in meters (achan)
tmp <- raster("achan.asc")
tmp <- as(tmp, "SpatialGridDataFrame")
grids$achan <- tmp
# Topographic Wetness Index (twi)
tmp <- raster("twi.asc")

```

```

tmp <- as(tmp, "SpatialGridDataFrame")
grids$twi <- tmp
# Aridity
aridity <- raster("meanAridity1.tif")
aridity[aridity > 0.1] <- NA
# Evapotranspiration (ET)
ET <- raster("meanET1.tif")
ET[ET <= 0.1] <- NA
ET[ET > 10000] <- NA
tmp <- as(ET, "SpatialGridDataFrame")
names(tmp)[1] <- "ET"
grids$ET <- tmp
# Land Surface Temperature (LST)
LST <- raster("meanLST1.tif")
LST[LST <= 0.1] <- NA
tmp <- as(LST, "SpatialGridDataFrame")
names(tmp)[1] <- "LST"
grids$LST <- tmp
# Load required packages
library(raster)
library(ncdf4)
# Load NDVI raster stack
ndvi_stack <- stack("Botswana_NDVI_USGS.tif") # Your NDVI time series
nlayers(ndvi_stack) # Should be > 1 (e.g., 12 months, or 1982–2022)

# Convert raster stack to matrix: rows = pixels, columns = time
ndvi_matrix <- as.matrix(values(ndvi_stack)) # matrix: n_pixels x n_time
# Create time variable (e.g., 1 to number of layers)
time_vector <- 1:nlayers(ndvi_stack)
# Initialize F-value vector
F_values <- apply(ndvi_matrix, 1, function(ndvi_pixel) {
  if (all(is.na(ndvi_pixel))) return(NA)
  tryCatch({
    model <- lm(ndvi_pixel ~ time_vector)
    summary(model)$statistic[1] # Extract F value
  }, error = function(e) NA)
})
# Create empty raster for output
F_raster <- ndvi_stack[[1]] # use first layer as template
values(F_raster) <- F_values
# Save F-value raster
writeRaster(F_raster, "F.tif", overwrite = TRUE)
# Optional: Plot it
plot(F_raster, main = "F-values from NDVI ~ Time Regression")
### import trend results
Fvalue <- raster("R.tif")
# Fvalue[Fsig < 0.1] <- NA
Fvalue[Fvalue == -999] <- NA
tmp <- as(Fvalue, "SpatialGridDataFrame")
grids$F <- tmp
## modelling
x <- sample(1:(dim(Fvalue)[1] * dim(Fvalue)[2]), 5000)
Fmult <- as.data.frame(raster(grids$F))
slope <- as.data.frame(raster(grids$slope))
elev <- as.data.frame(raster(grids$elev))
twi <- as.data.frame(raster(grids$twi))
achan <- as.data.frame(raster(grids$achan))
aspect <- as.data.frame(raster(grids$aspect))
LST <- as.data.frame(raster(grids$LST))
ET <- as.data.frame(raster(grids$ET))
flowdir <- as.data.frame(raster(grids$flowdir))
tri <- as.data.frame(raster(grids$tri))
tpi <- as.data.frame(raster(grids$tpi))
roughness <- as.data.frame(raster(grids$roughness))
aridity <- as.data.frame(raster(grids$aridity))
# Combine selected variables for regression
sub <- as.data.frame(cbind(
  Fmult[x, 1], slope[x, 1], aspect[x, 1], flowdir[x, 1], aridity[x, 1],
  ET[x, 1], LST[x, 1], roughness[x, 1], elev[x, 1], twi[x, 1],

```

```

    achan[x, 1], tri[x, 1], tpi[x, 1]
  ))
# Assign column names
names(sub) <- c("Fmult", "slope", "aspect", "flowdir", "aridity",
               "ET", "LST", "roughness", "elev", "twi", "achan", "tri", "tpi")
# Remove rows with missing values
sub <- sub[complete.cases(sub), ]
# Regression using random forest
# Split into training and testing sets
tmp <- sample(nrow(sub), 2/3 * nrow(sub))
train <- sub[tmp, ]
test <- sub[-tmp, ]
# Load dplyr and ranger
library(ranger)
# Train random forest model
rf <- ranger(Fmult ~ ., importance = "impurity", data = train) # regression
# Predict on test data
pred <- predict(rf, data = test)
# Plot predictions vs actual values
plot(test$Fmult, pred$predictions)
# Correlation between predicted and actual
cor(test$Fmult, pred$predictions)
# Variable Importance# rf model with importance
rf <- ranger(Fmult ~ .,
             importance = "impurity",
             probability = TRUE,
             data = train)
# Extract importance as data frame
rf_imp <- as.data.frame(rf$variable.importance)
# Rename the importance column
names(rf_imp)[1] <- "importance"
# Move row names (variable names) into a column
library(tibble)
rf_imp <- rf_imp %>% rownames_to_column("variable")
# 10. Load auxiliary rasters
slope <- rast("slope.tif")
aspect <- rast("aspect.tif")
rough <- rast("roughness.tif")
flow <- rast("flowdir.tif")
tri <- rast("tri.tif")
tpi <- rast("tpi.tif")
achan <- rast("achan.asc")
twi <- rast("twi.asc")
arid <- rast("meanAridity1.tif")
et <- rast("meanET1.tif")
lst <- rast("meanLST1.tif")
elev <- rast("Botswana_Elevation_resample.tif")
# 11. Load F-value raster from DLM result (assumed as fval.tif for this script)
fval <- rast("fval.tif")
# 12. Stack all explanatory variables
env_stack <- c(slope, aspect, rough, flow, tri, tpi, achan, twi, arid, et, lst, elev)
# 13. Combine F-values and explanatory variables
full_stack <- c(fval, env_stack)
names(full_stack)[1] <- "fval"
# 14. Extract complete pixel values
vals <- na.omit(as.data.frame(values(full_stack)))
# 15. Sample training & testing
set.seed(123)
sample_index <- sample(1:nrow(vals), size=5000)
train <- vals[sample_index, ]
test <- vals[-sample_index, ]
# 16. Random Forest Regression
model_rf <- ranger(fval ~ ., data=train, importance="impurity")
# 17. Predict on test set
pred <- predict(model_rf, data=test)$predictions# 18. Plot and assess
plot(test$fval, pred, xlab="Observed F-value", ylab="Predicted F-value",
     main="Validation: Observed vs Predicted")
abline(0, 1, col="red")
# 19. Variable Importance

```

```

barplot(sort(model_rf$variable.importance, decreasing=TRUE),
  main="Variable Importance", horiz=TRUE, las=1)
# Save outputs (optional)
# writeRaster(env_stack, "explanatory_vars_stack.tif")
# write.csv(vals, "env_variables_with_fval.csv", row.names=FALSE)
# variable importance not recommendend to use 50000, use 5000
# direction of correlation
tmp2 <- cor(train2)
tmp2[is.na(tmp2)] <- 0
rf <- ranger(as.factor(pval) ~ ., importance = "impurity", probability = T, data = train2)
rf_imp <- as.data.frame(rf$variable.importance * sign(tmp2))
rf_imp <- as.data.frame(rf$variable.importance)
names(rf_imp) <- "importance"
rf_imp <- rf_imp %>% rownames_to_column("variable")
windows()
p <- ggplot(rf_imp, aes(x=reorder(variable, importance), y=importance, fill=importance))+
  geom_bar(stat="identity", position="dodge")+ coord_flip()+
  ylab("Variable Importance")+
  xlab("")+
  ggtitle("Information Value Summary")+
  guides(fill=F)+
  scale_fill_gradient(low="red", high="blue")
print(p)
# Accuracy assessment
table(test2$pval, pred$predictions)
Acc <- caret::confusionMatrix(as.factor(as.character(pred$predictions)), as.factor(as.character(test2$pval)), positive="0")
summary(acc)

```